PySpark cơ bản — Buổi 1/3

Lecture Note

RDD: giải đáp 19 câu hỏi thường gặp

Tài liệu này dùng thế nào? Mỗi câu hỏi mở đầu bằng một hộp **Trả lời ngắn** (đọc trong 30 giây), sau đó là phần giải thích chi tiết, hình vẽ, và ví dụ code.
Mọi output trong tài liệu là output thật, chạy trên Spark 3.4.4, Python 3.7.4, Java 17, chế độ `local[*]` (8 luồng CPU). Chạy lại trên máy khác, các giá trị như PID, cổng, mã ứng dụng, thời gian sẽ khác; các giá trị tính toán thì giống.
Ký hiệu [slide 3.19] trỏ tới slide tương ứng trong `Slide_Buoi1_v2/Slide_Buoi1_RDD_v2.pdf`.

Mục lục

Bản đồ các câu hỏi **5**

Phần I Mở đầu: bài thực hành và dữ liệu **6**

0 Mục đích bài thực hành **6**

0.1 Vị trí của buổi học trong học phần 6

0.2 Mục tiêu cụ thể 6

0.3 Tài liệu và quy trình thực hành đề xuất 6

1 Ý nghĩa các cột trong file `access.log` **7**

1.1 Giải phẫu một dòng log 7

1.2 Bảng tổng quan sáu cột 7

1.3 Từng cột chi tiết 8

1.4 Dữ liệu được sinh ra như thế nào? 9

1.5 So với log của một máy chủ web thật 10

1.6 Các cột được dùng ở đâu trong buổi học 11

Phần II Môi trường chạy và các đối tượng điểm vào **12**

2 JVM là gì và tại sao nó quan trọng với PySpark? **12**

2.1 JVM là gì? 12

2.2 PySpark và JVM: ai chạy ở đâu? 12

2.3 Tại sao JVM quan trọng? Năm hệ quả thực tế 13

3 Session theo nghĩa chung là gì? **14**

3.1 Session trong các hệ thống quen thuộc 14

3.2 SparkSession nhìn theo bốn đặc trưng 14

4 SparkSession.builder và SparkContext **15**

4.1 SparkContext — kết nối tới cluster 15

4.2 SparkSession — điểm vào hợp nhất 16

4.3 Builder: mẫu thiết kế “dựng từng bước” 16

4.4 `getOrCreate()` hoạt động thế nào? — thí nghiệm 17

4.5 Tóm tắt quan hệ 17

5 Ví dụ: SparkSession là điểm vào hợp nhất bao ngoài SparkContext **18**

6 Application, SparkContext, SparkSession: cái nào chứa cái nào? 19
6.1 Định nghĩa . . . 20
6.2 Hình: quan hệ bao chứa và số lượng . . . 20
6.3 Quy mô và vòng đời . . . 21
6.4 Kiểm chứng . . . 21

7 sc là gì? So sánh sc.accumulator, sc.setLogLevel, sc.parallelize 22
7.1 `sc` là gì? . . . 22
7.2 Ba lệnh, ba nhóm việc . . . 22

Phần III Dữ liệu phân tán và đơn vị thực thi 24

8 Mối liên hệ giữa partition và RDD 24
8.1 Năm thuộc tính định nghĩa một RDD . . . 24
8.2 Số partition thay đổi thế nào qua các phép toán . . . 24
8.3 Partitioner: khi RDD "biết" khoá nằm ở partition nào . . . 25
8.4 Mọi cơ chế quan trọng đều vận hành theo partition . . . 25

9 "Số task của một stage = số partition của stage đó" — mỗi task ứng với một partition? 26
9.1 Định nghĩa chính xác . . . 26
9.2 Kiểm chứng: mỗi task ứng với một partition . . . 26
9.3 Bảng đo: số task trong các tình huống khác nhau . . . 27
9.4 Bốn chi tiết tránh hiểu sai . . . 27

10 Mối quan hệ giữa task và executor 28
10.1 Hai khái niệm ở hai quy mô khác nhau . . . 28
10.2 Thí nghiệm: 2 executor thật, task chạy theo đợt . . . 28
10.3 Rút ra từ thí nghiệm . . . 29

11 Khái niệm parallelize và serialize 30
11.1 Song song hoá nói chung . . . 30
11.2 `sc.parallelize`: từ list trên Driver thành RDD . . . 30
11.3 Serialize và deserialize . . . 31
11.4 Spark serialize ở những đâu? . . . 32

Phần IV Phép toán và mô hình thực thi 34

12 `reduceByKey` là transformation, `reduce` là action; `sample` là transformation, `takeSample` là action 34

12.1 `reduceByKey` và `reduce` . . . 34
12.2 `sample` và `takeSample` . . . 35
12.3 Quy tắc nhận biết chung và các cặp dễ nhầm . . . 36

13 Wide–long table trong pandas và narrow–wide transformation trong PySpark 36
13.1 Wide table và long table . . . 37
13.2 Nhắc lại: narrow và wide transformation . . . 38
13.3 Mối liên hệ thật sự: phép chuyển đổi giữa hai dạng bảng . . . 38

14 Closure và biến chia sẻ: Accumulator, Broadcast 40
14.1 Closure trong Python . . . 40
14.2 Closure trong Spark: gửi bản sao tới từng task . . . 40
14.3 Accumulator — kênh một chiều từ executor về Driver . . . 41
14.4 Broadcast — gửi một giá trị lớn, chỉ đọc, một lần mỗi executor . . . 42
14.5 Tổng hợp ba cơ chế . . . 43

15 Bảng tra cứu đầy đủ: tác dụng và ví dụ của từng hàm 43
15.1 Transformation . . . 43
15.2 Action . . . 46
15.3 Bảng tóm tắt một trang . . . 48

Phần V Thực thi pipeline, quan sát và tính đúng 49

16 Slide 4.16 có phải là tổng hợp các slide trước? Phần lý thuyết của pipeline nằm ở đâu? 49
16.1 Ánh xạ từng bước: lý thuyết → slide con → dòng code trong 4.16 . . . 49
16.2 Hai điểm 4.16 *khác* với việc ghép nối đơn thuần . . . 50
16.3 Phần lý thuyết của pipeline nằm ở đâu? . . . 51

17 Spark UI và log là gì? 51
17.1 Spark UI . . . 51
17.2 Log . . . 52
17.3 Spark UI hay log? . . . 54

18 Spark UI qua hình chụp thực tế 54
18.1 Ứng dụng dùng để chụp . . . 54
18.2 Tab Jobs — danh sách job của ứng dụng . . . 55
18.3 Chi tiết một job — DAG Visualization . . . 55
18.4 Stage bị bỏ qua (skipped) . . . 56

18.5 Chi tiết một stage — DAG bên trong và danh sách task 57
18.6 Tab Storage — RDD đang được cache 58
18.7 Tab Executors — tài nguyên của ứng dụng 59
18.8 Tab Environment — cấu hình thực sự được áp dụng 59

19 Vì sao `reduce` với phép chia cho kết quả khác nhau khi đổi số partition? 60
19.1 Một ”phép chia” thực chất là năm phép chia 60
19.2 Theo dõi từng số: nằm ở tử số hay mẫu số? 60
19.3 Vì sao Spark không giữ nguyên thứ tự trái sang phải? 61
19.4 Cách tính đúng: biến đổi về phép toán có tính kết hợp 61
19.5 Mở rộng: ngay cả phép cộng số thực cũng không kết hợp tuyệt đối 61

Tài liệu tham khảo 62

Bản đồ các câu hỏi

Các câu hỏi được chia thành **năm phần** theo chủ đề và **sắp xếp theo thứ tự kiến thức xuất hiện trong bài học**: câu nào dựa trên nội dung được dạy ở slide sớm hơn thì đứng trước; trong cùng một mốc, câu nền tảng đứng trước câu vận dụng. Nhờ vậy có thể đọc lecture note *song song* với slide và notebook `buoi1_rdd.ipynb`. Một số câu vẫn nhắc tới câu ở phía sau (ví dụ Câu 7 nhắc tới Accumulator ở Câu 14) — đó là chỗ kiến thức được *báo trước*, sẽ được giải thích đầy đủ khi tới nơi.

Phần	Câu	Nội dung	Slide liên quan
I. Mở đầu	0	Mục đích bài thực hành	Mở đầu
	1	Ý nghĩa các cột trong `access.log`	Mở đầu, 2.3
II. Môi trường chạy và đối tượng điểm vào	2	JVM là gì, vì sao quan trọng	3.1, 3.2
	3	Session theo nghĩa chung	3.4
	4	`SparkSession.builder` và `SparkContext`	3.4, 4.1, 4.2
	5	Ví dụ: SparkSession là điểm vào hợp nhất	3.4
	6	Application, SparkContext, SparkSession	3.4, 3.20
	7	`sc` là gì; `accumulator`, `setLogLevel`, `parallelize`	4.1 – 4.3
III. Dữ liệu phân tán và đơn vị thực thi	8	Partition và RDD	3.5 – 3.7
	9	Số task = số partition	3.3, 3.6
	10	Task và executor	3.1, 3.3, 3.7
	11	`parallelize` và serialize	4.3, 4.4, 3.19
IV. Phép toán và mô hình thực thi	12	`reduceByKey`/`reduce`, `sample`/`takeSample`	3.10
	13	Wide–long table và narrow–wide transformation	3.11, phụ lục A
	14	Closure, Accumulator, Broadcast	3.19
	15	Bảng tra cứu đầy đủ các hàm	3.21, 4.4 – 4.15
V. Thực thi pipeline, quan sát và tính đúng	16	Slide 4.16 và lý thuyết của pipeline	2.2 – 2.5, 4.16
	17	Spark UI và log	4.19
	18	Spark UI qua hình chụp thực tế	4.19
	19	`reduce` với phép chia	4.22 – 4.25

PHẦN I

Mở đầu: bài thực hành và dữ liệu

Câu 0 – 1 · slide Mở đầu (mục tiêu, bài toán xuyên suốt)

Câu 0. Mục đích bài thực hành

Trả lời ngắn

Buổi 1 dạy **mô hình thực thi** của Spark thông qua API thấp nhất là RDD. Mục đích không phải là thuộc lòng các hàm, mà là hiểu *Spark chạy code của bạn như thế nào*: dữ liệu được chia ra sao, chạy ở đâu, khi nào phải di chuyển, và vì sao một cách viết lại nhanh hơn cách khác. Mọi thứ học ở đây vẫn đúng khi sang DataFrame (Buổi 2) và chuẩn bị dữ liệu (Buổi 3).

0.1 Vị trí của buổi học trong học phần



RDD là lớp nền mà mọi API cao hơn của Spark được xây dựng trên đó. Học RDD trước giống như học cách một câu SQL được thực thi trước khi học tối ưu truy vấn: khi đã thấy tận mắt partition, shuffle, stage, ta mới hiểu được vì sao DataFrame nhanh và khi nào nó vẫn chậm.

0.2 Mục tiêu cụ thể

Sau bài thực hành, sinh viên có thể:

1. **Giải thích** kiến trúc Driver–Executor và vai trò của `SparkSession`/`SparkContext`.
2. **Mô tả** một bài toán dưới dạng pipeline các bước biến đổi, và chỉ ra bước nào cần di chuyển dữ liệu.
3. **Phân biệt** transformation và action, narrow và wide, và dự đoán số job, stage, task *trước khi chạy*.
4. **Kiểm chứng** dự đoán bằng `toDebugString()`, Spark UI và log.
5. **Viết đúng** phép gộp (tính kết hợp, giao hoán) và **viết nhanh** (lọc sớm, bớt cột, gộp cục bộ), có số liệu đo làm bằng chứng.

0.3 Tài liệu và quy trình thực hành đề xuất

Tài liệu	Dùng để
`buoi1_rdd.ipynb`	Notebook bài giảng: đọc lý thuyết trước mỗi ô code, chạy từng ô.
`buoi1_toiuu.ipynb`	Notebook thực nghiệm: 10 thí nghiệm đo hiệu năng và tính đúng đắn.
`Slide_Buoi1_RDD_v2.pdf`	80 slide: nguyên lý → pipeline → khái niệm → code → tối ưu.
Lecture note này	Giải đáp các câu hỏi sâu hơn, dùng khi ôn tập hoặc khi gặp điểm chưa rõ.

1. Chạy `./start_jupyter.sh`, mở notebook, chọn kernel **PySpark (Buoi 1-3)**.

2. Chạy `buoi1_rdd.ipynb` từng ô. Ở các ô có ghi chú "đoán trước", hãy **viết dự đoán ra giấy** trước khi chạy, đặc biệt là ô `reduce` với phép chia.
3. Trong lúc chạy, mở **Spark UI** (địa chỉ in ra ở ô đầu tiên) và quan sát tab *Jobs*, *Stages* (Câu 17).
4. Chạy `buoi1_toiuu.ipynb` (khoảng 3 phút), đối chiếu số liệu với Phần 5 của slide.
5. Trả lời 8 câu hỏi ôn tập ở cuối slide; làm bài tập về nhà (tính min/max latency theo endpoint).

Bài toán xuyên suốt

Dữ liệu là file `access.log` (6000 dòng, định dạng `LEVEL ip METHOD endpoint status latency`). Câu hỏi: *với mỗi endpoint, có bao nhiêu request lỗi và latency trung bình của chúng là bao nhiêu?* Hầu hết ví dụ trong tài liệu này dùng dữ liệu đó.

Câu 1. Ý nghĩa các cột trong file `access.log`

Trả lời ngắn

Mỗi dòng của `access.log` ghi lại **một request HTTP** mà một máy khách gửi tới một hệ thống web API. Dòng gồm **6 trường cách nhau bởi dấu cách**: **LEVEL** (mức độ nghiêm trọng của sự kiện), **ip** (địa chỉ máy gửi request), **METHOD** (phương thức HTTP), **endpoint** (đường dẫn API được gọi), **status** (mã trạng thái HTTP trả về) và **latency** (thời gian phản hồi, mili giây). File có 6000 dòng, 251.876 byte, và là **dữ liệu tổng hợp** sinh bằng `gen_data.py` với seed cố định, được thiết kế để minh hoạ *data skew* và có sẵn các dòng lỗi.

1.1 Giải phẫu một dòng log



Dòng đầu tiên của file. `r[i]` là vị trí của trường trong tuple mà hàm `parse` trả về [slide 4.5]: `('ERROR', '10.0.3.185', 'POST', '/api/search', 502, 358)`.

1.2 Bảng tổng quan sáu cột

	Cột	Ý nghĩa	Kiểu sau parse	Giá trị trong file (6000 dòng)
`r[0]`	LEVEL	mức độ nghiêm trọng của sự kiện	`str`	3 giá trị: `INFO` 87,6%, `WARN` 6,4%, `ERROR` 6,1%
`r[1]`	ip	địa chỉ IPv4 của máy khách	`str`	dạng `10.0.X.Y`; 2321 địa chỉ khác nhau
`r[2]`	METHOD	phương thức HTTP	`str`	`POST` 3068, `GET` 2932
`r[3]`	endpoint	đường dẫn API được gọi	`str`	6 giá trị, lệch mạnh: `/api/search` 54,9%
`r[4]`	status	mã trạng thái HTTP	`int`	6 giá trị: 200, 400, 404, 500, 502, 503
`r[5]`	latency	thời gian phản hồi (ms)	`int` (bỏ đuôi `ms`)	5 – 900 ms, trung bình 451,9

1.3 Từng cột chi tiết

LEVEL — mức độ nghiêm trọng

Cùng ý nghĩa với các mức log của Spark ở Câu 17: `INFO` là diễn biến bình thường, `WARN` là bất thường nhưng hệ thống vẫn chạy, `ERROR` là lỗi. Trong file này, LEVEL **được suy ra hoàn toàn từ status** — bảng chéo cho thấy mỗi mức ứng với đúng một nhóm mã:

```
status   200  400  404  500  502  503
level
ERROR      0    0    0  119  135  112      <- ERROR  = loi phia MAY CHU (5xx)
INFO    5253    0    0    0    0    0      <- INFO   = thanh cong (2xx)
WARN       0  196  185    0    0    0      <- WARN   = loi phia MAY KHACH (4xx)
```

Vì vậy, trong bài toán xuyên suốt, lọc `level == "ERROR"` tương đương lọc `status >= 500` — và lọc được ngay trên chuỗi thô bằng `startswith("ERROR")` vì LEVEL là từ đầu tiên của dòng [slide 5.3].

ip — địa chỉ máy khách

Địa chỉ IPv4 của máy gửi request. Mọi địa chỉ có dạng `10.0.X.Y` với $X \in [0, 9]$, $Y \in [1, 254]$ — thuộc dải **địa chỉ riêng** `10.0.0.0/8` (RFC 1918), tức là máy trong mạng nội bộ, không phải địa chỉ Internet công cộng. Có $10 \times 254 = 2540$ địa chỉ khả dĩ; file chứa **2321** địa chỉ khác nhau, trung bình mỗi địa chỉ xuất hiện 2,6 lần, 587 địa chỉ chỉ xuất hiện một lần, nhiều nhất là 8 lần.

Dùng trong buổi học: ví dụ `distinct` (cặp endpoint–IP), bảng tra cứu IP → vùng mạng với `broadcast` và `mapPartitions` (Câu 14, slide 5.8), và ví dụ ẩn danh IP bằng SHA-256 (slide 5.7). Octet thứ ba X được dùng làm "vùng mạng".

METHOD — phương thức HTTP

Phương thức	Ý nghĩa trong HTTP	Ví dụ thường gặp
`GET`	**lấy** dữ liệu, không thay đổi gì trên máy chủ	xem sản phẩm, tìm kiếm
`POST`	**gửi** dữ liệu để máy chủ xử lý / tạo mới	đăng nhập, thêm vào giỏ, thanh toán

endpoint — đường dẫn API

Phần đường dẫn của URL, xác định chức năng nào của hệ thống được gọi. Đây là cột dùng làm **khoá** trong `reduceByKey` của bài toán xuyên suốt.

Endpoint	Chức năng (diễn giải)	Số dòng	Tỉ lệ
`/api/search`	tìm kiếm sản phẩm	3296	54,9%
`/api/product`	xem chi tiết một sản phẩm	1050	17,5%
`/api/cart`	thao tác với giỏ hàng	709	11,8%
`/api/login`	đăng nhập	487	8,1%
`/api/checkout`	thanh toán đơn hàng	328	5,5%
`/api/admin`	chức năng quản trị	130	2,2%

Phân bố **lệch mạnh có chủ đích**: `/api/search` nhiều gấp 25 lần `/api/admin`. Sau shuffle theo endpoint, partition giữ `/api/search` phải xử lý hơn nửa dữ liệu — một ví dụ **data skew** nhìn thấy được [slide 3.14].

status — mã trạng thái HTTP

Con số máy chủ trả về cho biết request thành công hay thất bại, và lỗi thuộc về phía nào. Chữ số đầu cho biết nhóm:

Mã	Tên chuẩn	Ý nghĩa	Số dòng
200	OK	thành công	5253
400	Bad Request	request sai định dạng/thiếu tham số — lỗi **máy khách**	196
404	Not Found	tài nguyên không tồn tại — lỗi **máy khách**	185
500	Internal Server Error	lỗi bên trong ứng dụng — lỗi **máy chủ**	119
502	Bad Gateway	máy chủ trung gian nhận phản hồi hỏng từ máy chủ phía sau	135
503	Service Unavailable	máy chủ quá tải hoặc đang bảo trì	112

latency — thời gian phản hồi

Khoảng thời gian từ lúc máy chủ nhận request tới lúc trả phản hồi, tính bằng **mili giây**, ghi kèm đuôi `ms` (ví dụ `358ms`) — vì vậy phải cắt đuôi rồi ép kiểu: `int(c[5][:-2])`. Đây là cột số duy nhất cần tính toán (tổng, trung bình) trong bài toán xuyên suốt.

```
min 5   max 900   mean 451.88   median 451   p95 856   std 259.5
so dong theo khoang: (0, 300] 1969 | (300, 600] 2046 | (600, 900] 1985     <- gan nhu deu
```

1.4 Dữ liệu được sinh ra như thế nào?

Đọc mã sinh dữ liệu là cách chắc chắn nhất để hiểu các cột. Trích từ file `gen_data.py`, nằm cùng thư mục với dữ liệu:

`Slides_Tuan4_Phan3_XuLyTheoLo/data/gen_data.py`

```
random.seed(20260921)              # co dinh: moi sinh vien ra cung mot file
ENDPOINT = [("/api/search", 0.55), ("/api/product", 0.18), ("/api/cart", 0.12),
            ("/api/login", 0.08), ("/api/checkout", 0.05), ("/api/admin", 0.02)]

for _ in range(6000):
    ep = chon_endpoint()                                  # theo ti le o tren
    r = random.random()
    if r < 0.06:   code, muc = random.choice([500, 502, 503]), "ERROR"
    elif r < 0.12: code, muc = random.choice([400, 404]), "WARN"
    else:          code, muc = 200, "INFO"
    f.write("%s 10.0.%d.%d %s %s %d %dms\n" % (
        muc, random.randint(0, 9), random.randint(1, 254),  # ip
        random.choice(["GET", "POST"]), ep, code,          # method, endpoint, status
        random.randint(5, 900)))                           # latency: deu tu 5 den 900 ms
```

Cột	Quy tắc sinh	Kiểm chứng trên file
endpoint	theo tỉ lệ 55/18/12/8/5/2%	54,9 / 17,5 / 11,8 / 8,1 / 5,5 / 2,2%
LEVEL, status	6% lỗi 5xx, 6% lỗi 4xx, **như nhau cho mọi endpoint**	tỉ lệ `ERROR` theo endpoint: 4,6% – 9,2%
METHOD	50/50, **độc lập** với endpoint	% GET theo endpoint: 47,7% – 51,4%
latency	đều trên [5, 900], **độc lập** với mọi cột	trung bình lý thuyết 452,5; thực tế 451,9; theo level: 430 – 454
ip	đều trên 2540 địa chỉ	2321 địa chỉ xuất hiện

Hệ quả khi đọc kết quả phân tích

Vì các cột được sinh **độc lập**, mọi khác biệt giữa các nhóm trong file này là **nhiễu ngẫu nhiên**, không phải quy luật:

- `/api/admin` có tỉ lệ lỗi 9,2% so với 5,9% của `/api/search` — nhưng `/api/admin` chỉ có 130 dòng (12 lỗi), mẫu quá nhỏ; xác suất sinh lỗi của hai endpoint là như nhau.
- Kết quả bài toán xuyên suốt: latency trung bình của request lỗi là 341,2 ms ở `/api/admin` (12 request) và 506,1 ms ở `/api/checkout` (15 request). Chênh lệch này **không** nói lên endpoint nào chậm hơn.

Bài học chung cho phân tích dữ liệu lớn: trước khi kết luận "nhóm A khác nhóm B", hãy xem mỗi nhóm có bao nhiêu mẫu.

1.5 So với log của một máy chủ web thật

Log thật của Nginx hay Apache (định dạng *Combined Log Format*) có nhiều trường hơn. Một dòng minh hoạ định dạng đó:

```
10.0.3.185 - - [28/Sep/2026:11:04:56 +0700] "POST /api/search?q=laptop HTTP/1.1" 502 157 "https://shop.example/" "
    Mozilla/5.0 ..."
```

Trường của log thật	Có trong `access.log`?	Nếu thiếu thì không làm được
ip, method, đường dẫn, status	có	—
thời gian phản hồi	có (latency)	—
thời điểm (timestamp)	**không**	phân tích theo thời gian: lưu lượng theo giờ, phát hiện đỉnh tải, sự cố lúc nào
tham số truy vấn (`?q=...`)	không	thống kê từ khoá tìm kiếm
kích thước phản hồi (byte)	không	ước lượng băng thông
referrer, user agent	không	nguồn truy cập, loại thiết bị/trình duyệt
mức LEVEL	có (thêm vào)	— (log web thật thường không có; suy ra từ status)

Định dạng rút gọn giúp tập trung vào cơ chế của Spark. Với log thật, bước 2 tách trường sẽ phức tạp hơn nhiều (trường có dấu cách bên trong, nằm trong ngoặc kép) — thường dùng biểu thức chính quy thay cho `split()`.

1.6 Các cột được dùng ở đâu trong buổi học

Cột	Vai trò
LEVEL	điều kiện lọc `ERROR` (bước 3); lọc sớm trên chuỗi thô (Nguyên tắc 1)
ip	`distinct` cặp (endpoint, IP) — TN3; bảng tra cứu vùng mạng — TN7, Câu 14; ẩn danh bằng SHA-256 — TN6
METHOD	điều kiện lọc phụ (`ERROR` + `GET`) — TN8, slide 5.10
endpoint	**khoá** của `reduceByKey`, `groupByKey`, `countByKey` (bước 4 5); minh hoạ skew
status	lọc lỗi 5xx (`status >= 500`) — TN5; `countByValue` theo mã — slide 4.14
latency	**giá trị** để tính tổng và trung bình (bước 5); ví dụ `reduce` trung bình sai/đúng — slide 4.24, 4.25

PHẦN II

Môi trường chạy và các đối tượng điểm vào

Câu 2 – 7 · slide 3.1 – 3.4, 4.1 – 4.2: Driver/Executor, JVM, SparkSession, SparkContext

Câu 2. JVM là gì và tại sao nó quan trọng với PySpark?

Trả lời ngắn

JVM = Java Virtual Machine (máy ảo Java). Spark được viết bằng Scala và chạy **bên trong JVM**; PySpark chỉ là lớp vỏ Python điều khiển JVM đó. Vì vậy JVM quyết định: Spark có khởi động được không (`JAVA_HOME`), có bao nhiêu bộ nhớ (heap JVM), và vì sao code Python trên RDD chậm hơn DataFrame (dữ liệu phải qua lại giữa Python và JVM).

2.1 JVM là gì?

Định nghĩa

Java Virtual Machine là một chương trình mô phỏng một "máy tính" chuẩn hoá, chạy mã **bytecode** (`.class`, `.jar`). Trình biên dịch Java/Scala/Kotlin dịch mã nguồn ra bytecode; JVM trên từng hệ điều hành thực thi bytecode đó — nguyên tắc *"write once, run anywhere"*.

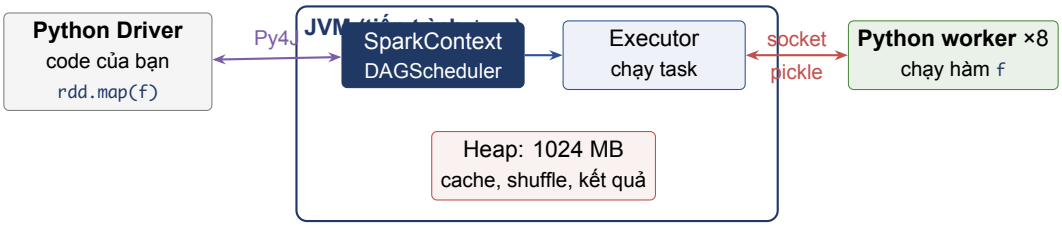
Những thành phần của JVM mà người dùng Spark cần biết:

- **Heap**: vùng bộ nhớ chứa mọi đối tượng. Kích thước tối đa được đặt khi JVM khởi động (`-Xmx`) và **không đổi được** khi đang chạy.
- **Garbage Collector (GC)**: tự động thu hồi bộ nhớ không dùng. GC có thể tạm dừng chương trình (*GC pause*).
- **JIT compiler**: biên dịch đoạn bytecode chạy nhiều thành mã máy — vì vậy lần chạy đầu thường chậm hơn các lần sau.
- **JDK / JRE**: bộ công cụ phát triển / môi trường chạy. Spark cần một JDK (Spark 3.4 hỗ trợ Java 8, 11, 17).

2.2 PySpark và JVM: ai chạy ở đâu?

Khi chạy `SparkSession.builder...getOrCreate()` trong notebook, trên máy này có **ba loại tiến trình**. Danh sách dưới đây lấy từ lệnh `ps` ngay sau khi chạy một job:

```
11156  python (kernel Jupyter)            <- Driver phia Python: code cua ban
11160  .../jdk-17.0.20.1+1/.../bin/java -cp .../pyspark/conf:...   <- JVM: Driver + Executor (
    local mode)
11190  python -m pyspark.daemon            <- tien trinh tao Python worker
11193  python -m pyspark.daemon            <- 8 Python worker (so worker pid khac nhau: 8)
 ...   (tong cong 9 tien trinh pyspark.daemon)
```



- **Py4J**: thư viện cho phép Python gọi phương thức của đối tượng Java. Mỗi lệnh như `sc.textFile` hay `rdd.count` thực chất là một lời gọi sang JVM.
- **Python worker**: JVM không chạy được hàm Python. Khi gặp `map(f)`, executor gửi dữ liệu (đã pickle) qua socket cho Python worker, worker chạy `f` rồi gửi kết quả về.
- Trong chế độ `local[*]`, Driver và Executor nằm **chung một JVM**; trên cluster, mỗi executor là một JVM riêng trên một máy.

Có thể hỏi trực tiếp JVM từ Python thông qua cổng Py4J:

```
jvm = sc._jvm                    # cong Py4J (API noi bo, chi dung de tim hieu)
print(jvm.java.lang.System.getProperty("java.version"))
print(jvm.java.lang.System.getProperty("java.vm.name"))
print(jvm.java.lang.Runtime.getRuntime().maxMemory() // 2**20, "MB")
```

```
17.0.20.1
OpenJDK 64-Bit Server VM
1024 MB          <- heap toi da = spark.driver.memory mac dinh (1g)
```

2.3 Tại sao JVM quan trọng? Năm hệ quả thực tế

1. **Không có JVM thì không có Spark.** Thiếu `JAVA_HOME` hoặc Java sai phiên bản, `getOrCreate()` báo: *"Java gateway process exited before sending its port number"* — Python không khởi động được JVM. Kernel `pyspark-buoi` của khoá học đặt sẵn `JAVA_HOME` trỏ tới JDK 17.
2. **Bộ nhớ của Spark là heap của JVM.** `spark.driver.memory`, `spark.executor.memory` chính là `-Xmx` của JVM. Vì heap không đổi được khi đang chạy, các cấu hình này phải đặt **trước khi JVM khởi động** (xem Câu 4). Lỗi hết bộ nhớ là lỗi của JVM: `java.lang.OutOfMemoryError`.
3. **Chi phí qua lại Python ↔ JVM.** Mỗi phần tử đi qua hàm Python phải được pickle, gửi qua socket, unpickle, xử lý, rồi làm ngược lại. Đây là lý do:
   - RDD với hàm Python chậm hơn RDD viết bằng Scala;
   - cache trong PySpark được lưu ở dạng byte đã pickle (`Memory Serialized`) [slide 4.21];
   - DataFrame nhanh hơn: phép toán được thực thi **ngay trong JVM**, không đi qua Python.
4. **Một JVM chỉ có một SparkContext.** Muốn tạo SparkContext mới (ví dụ đổi `master`) phải `stop()` cái cũ.
5. **Lỗi thường là lỗi Java.** Khi một task hỏng, Python nhận `Py4JJavaError` kèm một *stack trace* Java dài; nguyên nhân thật (lỗi trong hàm Python của bạn) nằm ở đoạn `PythonException: Traceback ...` bên trong.

Một chi tiết thú vị: PYTHONHASHSEED

Python ngẫu nhiên hoá hàm `hash()` của chuỗi mỗi lần khởi động. Nếu mỗi Python worker băm chuỗi khác nhau, cùng một khoá sẽ bị gửi tới các partition khác nhau khi shuffle — kết quả sai. Vì vậy PySpark đặt `PYTHONHASHSEED=0` cho mọi worker. Gọi `portable_hash("a")` trên Driver (không có biến này) sẽ báo lỗi *"Randomness of hash of string should be disabled via*

PYTHONHASHSEED".

Câu 3. Session theo nghĩa chung là gì?

Trả lời ngắn

Session (phiên làm việc) là một khoảng thời gian tương tác **có trạng thái** giữa một bên sử dụng và một hệ thống: có lúc **bắt đầu**, lúc **kết thúc**, và trong khoảng đó hệ thống **nhớ** những gì thuộc về bên sử dụng (danh tính, cấu hình, dữ liệu tạm, tài nguyên đã cấp). `SparkSession` đúng là như vậy: bắt đầu bằng `getOrCreate()`, giữ cấu hình, bảng tạm, hàm tự định nghĩa, và kết thúc bằng `stop()`.

3.1 Session trong các hệ thống quen thuộc

Hệ thống	Bắt đầu	Trạng thái được giữ	Kết thúc
Website	đăng nhập	mã phiên trong cookie, giỏ hàng, quyền truy cập	đăng xuất, hết hạn
Cơ sở dữ liệu	mở kết nối	giao dịch đang mở, bảng tạm, biến phiên	đóng kết nối
SSH / terminal	đăng nhập máy chủ	thư mục hiện tại, biến môi trường, tiến trình con	`exit`
Jupyter	khởi động kernel	mọi biến Python đã tạo trong notebook	Restart / Shutdown kernel
`requests.Session`	tạo đối tượng	cookie, header, kết nối TCP dùng lại	`close()`
SparkSession	`getOrCreate()`	cấu hình SQL, bảng tạm, UDF, executor đã cấp	`spark.stop()`

Bốn đặc trưng chung của một session

1. **Vòng đời**: có điểm bắt đầu và kết thúc rõ ràng.
2. **Trạng thái**: hệ thống nhớ thông tin giữa các lần tương tác — không phải gửi lại mọi thứ ở mỗi yêu cầu.
3. **Cách ly**: trạng thái của phiên này không lẫn sang phiên khác.
4. **Tài nguyên**: phiên giữ tài nguyên (kết nối, bộ nhớ, tiến trình) và phải được **giải phóng** khi kết thúc.

3.2 SparkSession nhìn theo bốn đặc trưng

Thí nghiệm: tạo một session thứ hai bằng `newSession()` rồi so sánh.

```
s3 = spark.newSession()
print("newSession is spark:", s3 is spark, "| cung sc:", s3.sparkContext is sc)

spark.range(3).createOrReplaceTempView("bang_tam")          # bang tam trong 'spark'
print("spark thay:", [t.name for t in spark.catalog.listTables()])
print("s3 thay   :", [t.name for t in s3.catalog.listTables()])
```

```
spark.conf.set("spark.sql.shuffle.partitions", "8")        # cau hinh luc chay
print(spark.conf.get("spark.sql.shuffle.partitions"), s3.conf.get("spark.sql.shuffle.
    partitions"))
```

```
newSession is spark: False | cung sc: True
spark thay: ['bang_tam']
s3 thay   : []                <- bang tam cua session nay KHONG thay o session kia
8 200                         <- cau hinh SQL cung tach rieng (200 la mac dinh)
```

Đặc trưng	Ở SparkSession
Vòng đời	`SparkSession.builder...getOrCreate()` → … → `spark.stop()`
Trạng thái	cấu hình SQL (`spark.conf`), bảng tạm và view, hàm UDF đã đăng ký, cache của DataFrame
Cách ly	mỗi session (kể cả từ `newSession()`) có bảng tạm và cấu hình SQL riêng
Tài nguyên	**dùng chung** một SparkContext: executor, bộ nhớ, cache RDD, Spark UI

Phân biệt ba khái niệm hay bị lẫn

- **Application** (ứng dụng Spark): một chương trình, có một `applicationId`, một Spark UI.
- **SparkContext**: **một** cho mỗi application — đại diện cho tài nguyên cluster.
- **SparkSession**: có thể **nhiều** trong một application, dùng chung SparkContext nhưng cách ly trạng thái SQL.

Trong notebook, "kết thúc session" theo nghĩa giải phóng tài nguyên là `spark.stop()` — nó dừng cả SparkContext bên dưới.

Câu 4. SparkSession.builder và SparkContext

Trả lời ngắn

`SparkContext` là **kết nối tới cluster** (API cấp thấp, dùng cho RDD). `SparkSession` là **điểm vào hợp nhất** bao bên ngoài một SparkContext, thêm SQL, DataFrame, catalog. `SparkSession.builder` là **đối tượng dựng** (builder) dùng để khai báo cấu hình rồi gọi `getOrCreate()`: nếu đã có session thì **trả lại session cũ** và bỏ qua các cấu hình không đổi được lúc chạy.

4.1 SparkContext — kết nối tới cluster

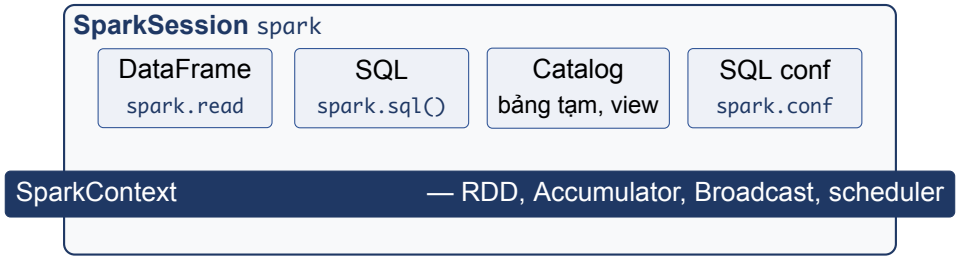
Định nghĩa

`SparkContext` (thường đặt tên biến `sc`) đại diện cho **một ứng dụng Spark đang kết nối với cluster manager**. Nó xin tài nguyên (executor), giữ cấu hình, lập lịch job, và là "nhà máy" tạo RDD, Accumulator, Broadcast.

Thành phần	Ý nghĩa
`sc.textFile`, `sc.parallelize`	tạo RDD
`sc.accumulator`, `sc.broadcast`	tạo biến chia sẻ (Câu 14)
`sc.master`, `sc.appName`	cluster manager và tên ứng dụng: `local[*]`, `LectureNote`
`sc.defaultParallelism`	số partition mặc định: `8`
`sc.applicationId`	mã ứng dụng, xuất hiện trong UI và log: `local-1790550242796`
`sc.uiWebUrl`	địa chỉ Spark UI (Câu 17)
`sc.setLogLevel(...)`, `sc.setJobGroup(...)`	mức log; gắn nhãn job
`sc.getConf()`	cấu hình đang áp dụng
`sc.stop()`	kết thúc ứng dụng, giải phóng executor

Trước Spark 2.0, ta tạo trực tiếp: `sc = SparkContext(master="local[*]", appName="App")`. Cách này vẫn chạy được, nhưng không có DataFrame/SQL.

4.2 SparkSession — điểm vào hợp nhất



4.3 Builder: mẫu thiết kế "dựng từng bước"

Mẫu thiết kế Builder

Builder là một mẫu thiết kế hướng đối tượng: thay vì một hàm khởi tạo nhận rất nhiều tham số, ta tạo một đối tượng trung gian, **gọi nối chuỗi** các phương thức để khai báo từng cấu hình, rồi gọi một phương thức cuối để **tạo ra đối tượng thật**. Mỗi phương thức cấu hình trả về chính builder, nên có thể viết nối tiếp.

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder                          # 1. lay doi tuong Builder
         .appName("LectureNote")                       # 2. khai bao cau hinh...
         .master("local[*]")
         .config("spark.ui.showConsoleProgress", "false")
         .getOrCreate())                               # 3. TAO (hoac lay lai) SparkSession
sc = spark.sparkContext
```

Phương thức của builder	Tác dụng
`.appName(name)`	tên ứng dụng (UI, log)
`.master(url)`	cluster manager: `local`, `local[4]`, `local[*]`, `yarn`, `spark://host:7077`, `k8s://...`
`.config(key, value)`	cấu hình bất kỳ, ví dụ `"spark.driver.memory", "4g"`
`.enableHiveSupport()`	dùng Hive metastore (Buổi 2 trở đi, nếu cần)
`.getOrCreate()`	trả về session đang hoạt động, hoặc tạo mới nếu chưa có

4.4 `getOrCreate()` hoạt động thế nào? — thí nghiệm

```
# Da co 'spark' (appName="LectureNote"). Goi builder lan nua voi cau hinh khac:
s2 = SparkSession.builder.appName("TenKhac").config("spark.driver.memory", "2g").getOrCreate()
print("s2 is spark:", s2 is spark)
print("appName van la:", sc.appName)
print("spark.driver.memory:", sc.getConf().get("spark.driver.memory", "(khong dat)"))
print("sparkContext is sc:", spark.sparkContext is sc)
```

```
s2 is spark: True                <- KHONG tao session moi
appName van la: LectureNote      <- appName moi bi bo qua
spark.driver.memory: (khong dat) <- 2g KHONG duoc ap dung
sparkContext is sc: True
```

Đồng thời log in ra cảnh báo:

```
WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
```

Cấu hình tĩnh và cấu hình lúc chạy

- **Cấu hình tĩnh** (static): đọc một lần khi JVM/SparkContext khởi động — `spark.master`, `spark.app.name`, `spark.driver.memory`, `spark.executor.memory`… Đặt sau đó **không có tác dụng**.
- **Cấu hình lúc chạy** (runtime SQL conf): đổi được bất cứ lúc nào qua `spark.conf.set(...)`, ví dụ `spark.sql.shuffle.partitions`.

Hệ quả trong Jupyter

Muốn đổi cấu hình tĩnh (bộ nhớ, master) trong notebook: gọi `spark.stop()` rồi tạo lại, hoặc chắc chắn nhất là **Restart Kernel** rồi chạy lại ô khởi tạo với cấu hình mới. Đặt mọi `.config(...)` quan trọng ngay trong ô đầu tiên của notebook.

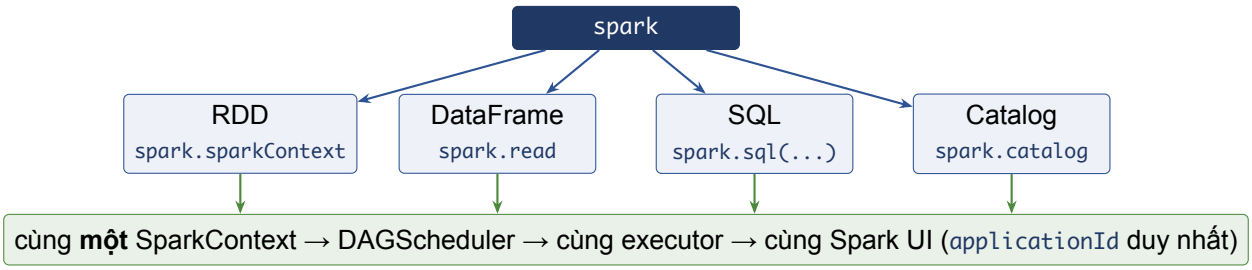
4.5 Tóm tắt quan hệ

	SparkContext	SparkSession
Có từ	Spark 1.x	Spark 2.0
Vai trò	kết nối tới cluster, lập lịch, tạo RDD	điểm vào cho DataFrame, SQL, catalog; chứa một SparkContext
Số lượng	**một** trên mỗi JVM	có thể **nhiều** (`newSession()`), dùng chung SparkContext (Câu 3)
Tạo bằng	`spark.sparkContext`	`SparkSession.builder ...getOrCreate()`
Dùng ở	Buổi 1 (RDD)	Buổi 2, 3 (DataFrame)

Câu 5. Ví dụ: SparkSession là điểm vào hợp nhất bao ngoài SparkContext

Trả lời ngắn

"Điểm vào hợp nhất" nghĩa là **chỉ cần một đối tượng** `spark` là dùng được **mọi** API: RDD (qua `spark.sparkContext`), DataFrame (`spark.read`), SQL (`spark.sql`), danh mục bảng (`spark.catalog`). "Bao bên ngoài" nghĩa là tất cả những API đó cuối cùng đều chạy trên **cùng một SparkContext**, cùng executor, cùng `applicationId`, cùng Spark UI. Ví dụ dưới đây giải bài toán xuyên suốt bằng cả ba cách từ **một** session.




```
print(spark.version, spark.conf.get("spark.app.name"), spark.sparkContext.applicationId)

# 1. RDD API -- qua spark.sparkContext
rdd = spark.sparkContext.textFile(f"{DATA}/access.log")
kq_rdd = sorted(rdd.map(lambda l: l.split()).filter(lambda c: c[0] == "ERROR")
                   .map(lambda c: (c[3], 1)).reduceByKey(add).collect())

# 2. DataFrame API -- qua spark.read, co schema
from pyspark.sql import functions as F
df = (spark.read.text(f"{DATA}/access.log")
           .select(F.split("value", " ").alias("c"))
           .select(F.col("c")[0].alias("level"), F.col("c")[3].alias("endpoint"),
                   F.regexp_replace(F.col("c")[5], "ms", "").cast("int").alias("latency")))
kq_df = sorted((r["endpoint"], r["n"]) for r in
               df.filter("level = 'ERROR'").groupBy("endpoint").agg(F.count("*").alias("n")).
    collect())

# 3. SQL -- dang ky bang tam trong catalog, roi truy van
df.createOrReplaceTempView("logs")
kq_sql = sorted(tuple(r) for r in spark.sql(
    "SELECT endpoint, COUNT(*) AS n FROM logs WHERE level = 'ERROR' GROUP BY endpoint").
    collect())

print(kq_rdd == kq_df == kq_sql, kq_sql)
print(spark.catalog.listTables())                                    # 4. Catalog
print(df.rdd.context is spark.sparkContext)                          # DataFrame -> RDD: cung sc
spark.sparkContext.parallelize([("search", 193), ("cart", 49)]).toDF(["endpoint", "so_loi"]).
    show()
```

```
3.4.4 LectureNote-App2 local-1790568564365
True [('/api/admin', 12), ('/api/cart', 49), ('/api/checkout', 15), ('/api/login', 29),
      ('/api/product', 68), ('/api/search', 193)]           <- 3 API, cung mot ket qua
[Table(name='logs', database=None, ..., tableType='TEMPORARY', isTemporary=True)]
```

```
True                                                    <- DataFrame va RDD dung chung
    SparkContext
+--------+------+
|endpoint|so_loi|
+--------+------+
|  search|   193|
|    cart|    49|
+--------+------+
```

Thêm gì so với SparkContext	Truy cập bằng	Trong ví dụ
DataFrame, schema	`spark.read`, `spark.createDataFrame`	`df` có schema `level: string, endpoint: string, latency: int`
SQL	`spark.sql(...)`	câu `SELECT ... GROUP BY` trên bảng `logs`
Catalog	`spark.catalog`	bảng tạm `logs` được liệt kê, loại `TEMPORARY`
Cấu hình SQL	`spark.conf`	`spark.conf.get("spark.app.name")`
Vẫn giữ RDD	`spark.sparkContext`	`rdd`, `parallelize(...).toDF(...)`

Liên hệ

Chuyển qua lại giữa hai thế giới rất dễ vì cùng một SparkContext: `df.rdd` cho RDD các `Row`; `rdd.toDF([...])` hoặc `spark.createDataFrame(rdd)` cho DataFrame. Đây là cầu nối từ Buổi 1 sang Buổi 2.

Câu 6. Application, SparkContext, SparkSession: cái nào chứa cái nào?

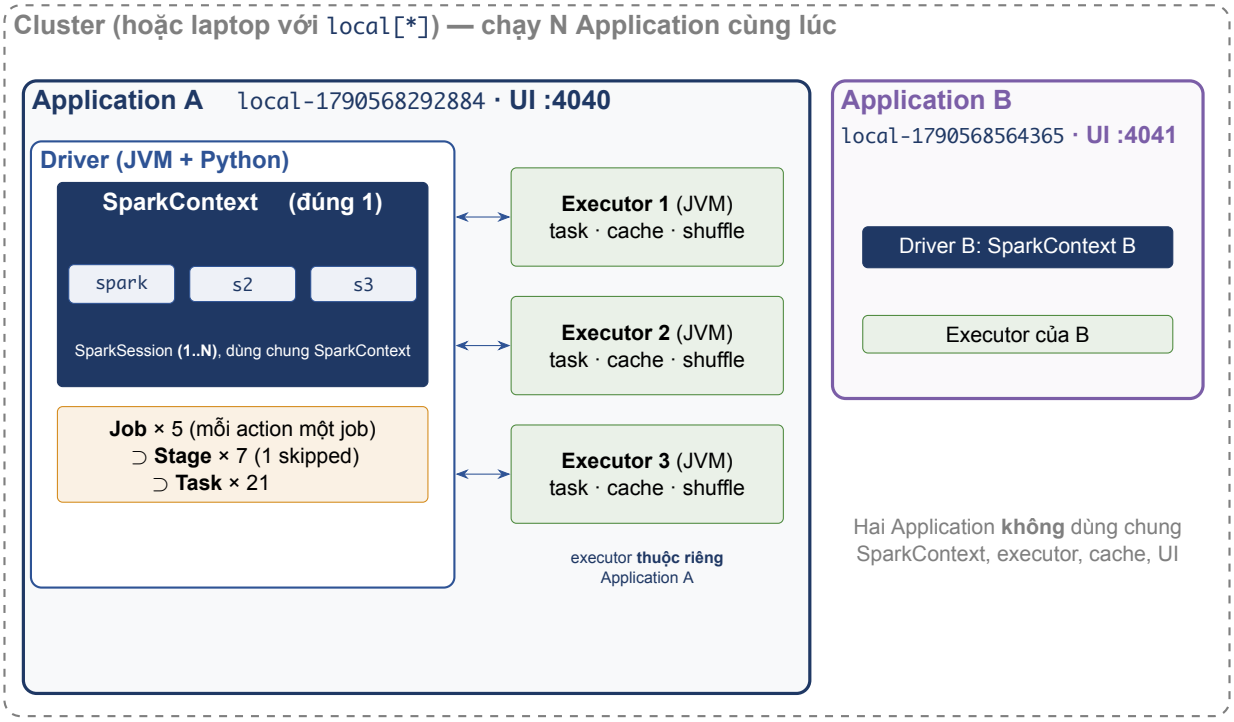
Trả lời ngắn

Từ lớn tới nhỏ: **Application** (một chương trình Spark đang chạy: 1 Driver + các executor của nó) **chứa đúng một SparkContext** (kết nối tới cluster, sở hữu executor và bộ lập lịch), SparkContext lại được **dùng chung bởi một hoặc nhiều SparkSession** (các ”phiên” SQL nhẹ, mỗi phiên có bảng tạm và cấu hình riêng). Công việc của Application được chia theo chiều khác: **Job** (mỗi action) $\supset$ **Stage** $\supset$ **Task**. Một cluster có thể chạy **nhiều Application** cùng lúc, mỗi cái có executor, `applicationId` và Spark UI riêng.

6.1 Định nghĩa

Khái niệm	Định nghĩa	Nhận diện
Application	Một chương trình người dùng chạy trên Spark: **một Driver** và **các executor được cấp riêng cho nó**. Bắt đầu khi SparkContext được tạo, kết thúc khi nó dừng.	`sc.applicationId`, một Spark UI (cổng 4040, 4041…)
SparkContext	Đối tượng trong Driver đại diện cho kết nối tới cluster: xin executor, lập lịch job, tạo RDD/broadcast/accumulator.	`spark.sparkContext`; **một** mỗi JVM
SparkSession	Điểm vào cho DataFrame/SQL; giữ cấu hình SQL, bảng tạm, UDF. Nhẹ, tạo thêm được bằng `newSession()`.	`spark`; nhiều phiên dùng chung một `sc`

6.2 Hình: quan hệ bao chứa và số lượng



Mã ứng dụng, cổng UI và số job/stage/task là số liệu thật của hai ứng dụng chạy đồng thời: Application A là `Buoi1-SparkUI-Demo` (Câu 18), Application B chạy trong một tiến trình Python khác. Ba executor là minh hoạ cho trường hợp cluster; ở chế độ `local[*]` mỗi Application chỉ có một executor tên `driver` nằm chung JVM với Driver (tab Executors, Câu 18).

6.3 Quy mô và vòng đời

Cấp	Số lượng	Sống bao lâu	Ví dụ Application A
Cluster	1	nhiều tháng, năm	laptop, `local[*]`, 8 luồng
Application	N trên một cluster	phút đến giờ (tới khi `stop()`)	`Buoi1-SparkUI-Demo`
SparkContext	**đúng 1** mỗi Application	bằng Application	`local-1790568292884`
SparkSession	1..N mỗi SparkContext	tuỳ ý, nhẹ	`spark`
Job	1 mỗi action	giây đến phút	5 job
Stage	số lần shuffle + 1 mỗi job	giây	7 stage (1 skipped)
Task	1 mỗi partition mỗi stage	mili giây đến giây	21 task

6.4 Kiểm chứng

Chạy một chương trình thứ hai trong lúc `Buoi1-SparkUI-Demo` vẫn đang chạy:

```
print("App2:", sc.uiWebUrl, sc.applicationId)
SparkContext(master="local[2]", appName="SC-thu-hai")          # thu tao SparkContext thu hai
print(SparkContext.getOrCreate() is sc)
s2, s3 = spark.newSession(), spark.newSession()
print({x.sparkContext.applicationId for x in (spark, s2, s3)})
spark.stop()
s3.range(3).count()                                            # dung session sau khi stop
```

```
App2: http://macbookpro:4041 local-1790568564365     <- App1 dang giu cong 4040, local
    -1790568292884
ValueError: Cannot run multiple SparkContexts at once; existing SparkContext(app=LectureNote-
    App2,
            master=local[*]) created by getOrCreate at verify_note2.py:13
True                                                  <- getOrCreate tra lai SparkContext dang
     co
{'local-1790568564365'}                               <- 3 session, MOT applicationId
AttributeError: 'NoneType' object has no attribute 'sc'   <- stop() dung SparkContext -> moi
    session hong
```

- Hai chương trình = hai Application: hai `applicationId`, hai cổng UI (4040, 4041), hai bộ executor.
- Trong một Application không tạo được SparkContext thứ hai.
- Nhiều SparkSession trong một Application đều mang cùng `applicationId` — chúng là ”phiên” *bên trong* Application.
- `spark.stop()` dừng SparkContext, tức là kết thúc Application — **mọi** session của nó không dùng được nữa.

Ví von dễ nhớ

Cluster là **toà nhà văn phòng**; mỗi Application là **một công ty** thuê văn phòng trong toà nhà (có phòng, nhân viên riêng — executor); SparkContext là **giám đốc điều hành** — mỗi công ty đúng một người; các SparkSession là **các phòng ban** dùng chung giám đốc và nhân viên

nhưng có hồ sơ, sổ sách riêng (bảng tạm, cấu hình SQL). Job, stage, task là **dự án, giai đoạn, đầu việc** mà công ty thực hiện.

Câu 7. `sc` là gì? So sánh `accumulator`, `setLogLevel`, `parallelize`

Trả lời ngắn

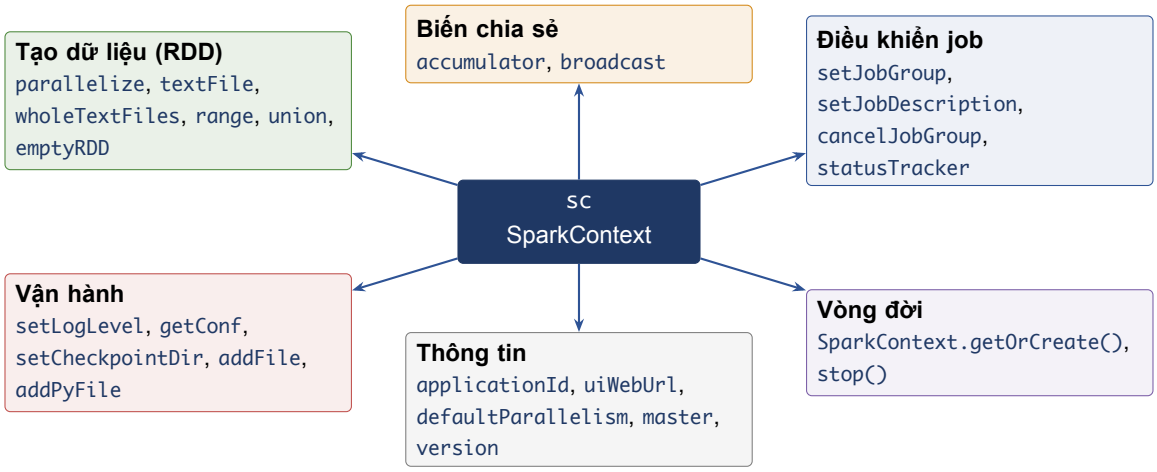
`sc` **không phải một từ khoá** của Spark — nó chỉ là **tên biến** (theo quy ước) giữ đối tượng **SparkContext** của ứng dụng, lấy bằng `sc = spark.sparkContext`. Ba lệnh trong câu hỏi đều là **phương thức của cùng một đối tượng** nhưng thuộc ba nhóm việc hoàn toàn khác nhau: `sc.parallelize` **tạo dữ liệu** (trả về RDD), `sc.accumulator` **tạo biến chia sẻ** (trả về Accumulator), `sc.setLogLevel` **chỉnh cách ứng dụng ghi log** (không trả về gì). Không lệnh nào trong ba lệnh này chạy job.

7.1 `sc` là gì?

```
sc = spark.sparkContext
print(type(sc).__module__ + "." + type(sc).__name__, "|", spark.sparkContext is sc)
print(len([m for m in dir(sc) if not m.startswith("_")]), "thuoc tinh/phuong thuc cong khai")
```

```
pyspark.context.SparkContext | True
55 thuoc tinh/phuong thuc cong khai
```

`sc` là một đối tượng Python thuộc lớp `pyspark.context.SparkContext`, đứng trên Driver, nói chuyện với JVM qua Py4J (Câu 2) và đại diện cho **toàn bộ Application** (Câu 6). Đặt tên `sc` chỉ là thói quen — viết `ctx = spark.sparkContext` cũng hoàn toàn được. Có thể hình dung `sc` như **bảng điều khiển trung tâm** của ứng dụng, với 55 nút bấm chia thành sáu nhóm:



7.2 Ba lệnh, ba nhóm việc

```
sc.setJobGroup("q15", "q15")
a = sc.accumulator(0)                          # bien chia se
r = sc.parallelize([1, 2, 3, 4, 5, 6], 3)      # du lieu
x = sc.setLogLevel("WARN")                     # van hanh
print(type(a).__name__, repr(a), "|", type(r).__name__, r.getNumPartitions(), "|", x)
print("so job da chay:", sc.statusTracker().getJobIdsForGroup("q15"))

r.foreach(lambda v: a.add(v))                  # ACTION: bay gio moi co job
```

```
print("a.value =", a.value, "| so job:", sc.statusTracker().getJobIdsForGroup("q15"))
sc.setLogLevel("CANH_BAO")                    # muc log khong hop le
```

```
Accumulator Accumulator<id=0, value=0> | RDD 3 | None
so job da chay: []                              <- ca ba lenh deu KHONG chay job
a.value = 21 | so job: [0]                      <- 1 + 2 + ... + 6 = 21, cong don tu 3 task
IllegalArgumentException: requirement failed: Supplied level CANH_BAO did not match one of:
ALL,DEBUG,ERROR,FATAL,TRACE,WARN,INFO,OFF
```

	`sc.parallelize(c, n)`	`sc.accumulator(v)`	`sc.setLogLevel(level)`
Nhóm việc	**tạo dữ liệu** phân tán	**tạo biến chia sẻ**	**vận hành**: chỉnh log
Trả về	`RDD`	`Accumulator`	`None`
Tác động lên	một tập dữ liệu mới (`n` partition)	một bộ đếm cộng dồn	toàn bộ ứng dụng (mọi log sau đó)
Có chạy job?	không (RDD lazy) — nhưng list được **serialize ngay** lúc gọi (Câu 11)	không	không
Dữ liệu đi đâu	Driver → executor	executor → Driver (`.value`)	không có dữ liệu
Dùng khi	dữ liệu nhỏ, thử nghiệm	đếm dòng hỏng, số liệu phụ	giảm log (`"WARN"`) hoặc gỡ lỗi (`"INFO"`)
Liên hệ	Câu 8, 11, 15	Câu 14	Câu 17

Vì sao ba thứ khác hẳn nhau lại ”ở chung” trong `sc`?

Vì chúng đều cần tới **một thứ** mà chỉ SparkContext có: kết nối tới các executor và JVM. `parallelize` cần phân phát dữ liệu tới executor; `accumulator` cần một kênh để executor gửi kết quả về; `setLogLevel` cần chỉnh bộ ghi log (Log4j) bên trong JVM. SparkContext là ”cửa” duy nhất tới những tài nguyên đó — vì vậy mỗi ứng dụng chỉ có **một** (Câu 6).

PHẦN III

Dữ liệu phân tán và đơn vị thực thi

Câu 8 – 11 · slide 3.3, 3.5 – 3.7, 4.3 – 4.4: RDD, partition, task, executor, tạo RDD

Câu 8. Mối liên hệ giữa partition và RDD

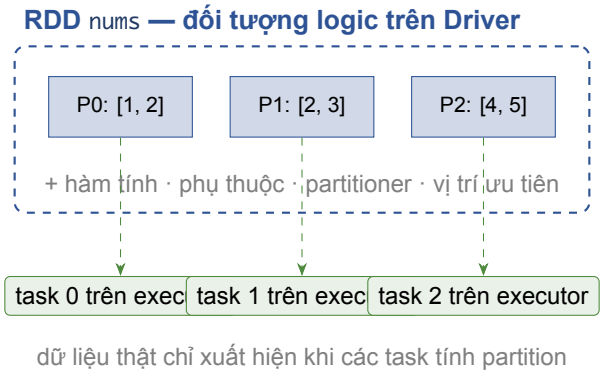
Trả lời ngắn

Partition là **một thành phần trong định nghĩa** của RDD. Một RDD *là* một danh sách partition, cộng với cách tính từng partition và thông tin về RDD cha. RDD là khái niệm **logic** (một tập dữ liệu, một công thức); partition là đơn vị **vật lý** (một mảnh nằm trên một executor, được một task tính ra). RDD không chứa dữ liệu — dữ liệu chỉ tồn tại khi các partition được tính (hoặc được cache).

8.1 Năm thuộc tính định nghĩa một RDD

Theo bài báo gốc (Zaharia và cộng sự, NSDI 2012) và mã nguồn `RDD.scala`, mỗi RDD được mô tả bởi năm thuộc tính:

	Thuộc tính	Ý nghĩa / cách xem trong PySpark
1	**Danh sách partition**	chia tập dữ liệu thành mảnh nào — `getNumPartitions()`, `glom()`
2	**Hàm tính mỗi partition**	từ partition cha tạo ra partition này như thế nào (hàm `map`, `filter`…)
3	**Danh sách phụ thuộc**	RDD cha nào, narrow hay wide — `toDebugString()` (lineage)
4	**Partitioner** (tuỳ chọn)	với pair RDD: khoá nào nằm ở partition nào — `rdd.partitioner`
5	**Vị trí ưu tiên** (tuỳ chọn)	partition nên chạy trên máy nào (gần dữ liệu, ví dụ block HDFS)



8.2 Số partition thay đổi thế nào qua các phép toán

```
nums = sc.parallelize([1, 2, 2, 3, 4, 5], 3)
print(nums.getNumPartitions(), nums.glom().collect())
print(nums.map(lambda x: x * 10).glom().collect())       # narrow: giu nguyen so partition
```

```
print(nums.filter(lambda x: x > 3).glom().collect())     # narrow: co the de lai partition
    RONG
```

```
3 [[1, 2], [2, 3], [4, 5]]
[[10, 20], [20, 30], [40, 50]]
[[], [], [4, 5]]              <- 3 partition, 2 cai rong -> van sinh 3 task
```

Phép toán	Số partition của RDD con
`map`, `filter`, `flatMap`, `mapValues`	bằng RDD cha (partition i của con chỉ phụ thuộc partition i của cha)
`union(a, b)`	tổng số partition của `a` và `b`
`reduceByKey`, `groupByKey`, `distinct`, `join`	`numPartitions` nếu truyền vào; không thì `spark.default.parallelism` hoặc bằng cha
`repartition(n)`, `coalesce(n)`	đúng `n`

8.3 Partitioner: khi RDD "biết" khoá nằm ở partition nào

```
from operator import add
from pyspark.rdd import portable_hash

pr = sc.parallelize([("a", 1), ("b", 2), ("a", 3), ("c", 4)], 2)
r = pr.reduceByKey(add)
print(pr.partitioner, "|", r.partitioner is not None, r.glom().collect())
print(r.mapValues(lambda v: v * 2).partitioner is not None)       # mapValues GIU partitioner
print(r.map(lambda kv: (kv[0], kv[1] * 2)).partitioner)           # map LAM MAT partitioner
print(sc.parallelize(list("abc")).map(lambda k: (k, portable_hash(k) % 2)).collect())
```

```
None | True [[('b', 2), ('c', 4)], [('a', 4)]]
True
None
[('a', 1), ('b', 0), ('c', 0)]      <- hash(khoa) % 2 -> dung partition cua tung khoa o tren
```

- Sau `reduceByKey`, RDD có **hash partitioner**: khoá `k` nằm ở partition `portable_hash(k) % n`. Dòng cuối kiểm chứng: `a` → 1, `b`, `c` → 0 — khớp với `glom()`.
- `mapValues` không đổi khoá nên **giữ** partitioner; `map` có thể đổi khoá nên Spark **xoá** partitioner.
- Giữ được partitioner giúp `join`/`reduceByKey` tiếp theo theo cùng khoá **không phải shuffle lại**. Vì vậy nên dùng `mapValues` thay cho `map` khi chỉ biến đổi giá trị.

8.4 Mọi cơ chế quan trọng đều vận hành theo partition

Cơ chế	Đơn vị là partition
Tính toán	1 task = 1 partition (Câu 9)
Cache	Spark cache **từng partition**; thiếu RAM thì một số partition không được cache, được tính lại khi cần
Phục hồi lỗi	mất executor → chỉ **tính lại các partition bị mất** theo lineage
Ghi file	`saveAsTextFile`: mỗi partition một file `part-0000i`
Shuffle	mỗi partition nguồn ghi một khối cho mỗi partition đích

Câu 9. "Số task của một stage = số partition của stage đó" — mỗi task ứng với một partition?

Trả lời ngắn

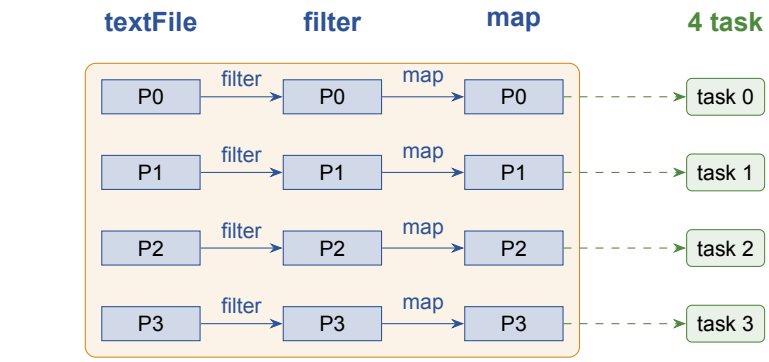
Đúng. Mỗi task xử lý **đúng một partition** — cụ thể là một partition của **RDD cuối cùng trong stage**. Stage có 4 partition thì sinh 4 task; task số i tính partition số i. Có bốn chi tiết cần nắm để không hiểu sai: (1) sau `coalesce`, một partition (và một task) có thể *đọc* nhiều partition cha; (2) `take(n)` chỉ chạy task cho vài partition đầu; (3) stage đã có kết quả shuffle sẵn sẽ bị *bỏ qua* (skipped), không sinh task; (4) một task có thể được *chạy lại* (attempt) khi lỗi, nhưng vẫn là một task của cùng partition.

9.1 Định nghĩa chính xác

Định nghĩa

- **Stage** là một chuỗi transformation narrow liền nhau, kết thúc tại một ranh giới shuffle hoặc tại action.
- **Task** là việc *áp dụng toàn bộ chuỗi của một stage lên một partition*. Task được định danh bởi cặp *(stage, chỉ số partition)*.
- Có hai loại task: **ShuffleMapTask** (stage kết thúc bằng shuffle — ghi file shuffle) và **ResultTask** (stage cuối — tính kết quả cho action).

Vì vậy: **số task của một stage = số partition của RDD cuối cùng trong stage đó**.



một stage — mỗi "hàng ngang" là một task, chạy trọn cả chuỗi

Chú ý: task là **một hàng ngang** xuyên qua cả stage, không phải một ô. Task 0 đọc partition 0 của file, lọc, map, rồi mới chuyển sang phần tử kế tiếp — đó là *pipelining*.

9.2 Kiểm chứng: mỗi task ứng với một partition

```
logs = sc.textFile(f"{DATA}/access.log", 4)
print(logs.getNumPartitions())
print(logs.mapPartitionsWithIndex(lambda i, it: [(i, sum(1 for _ in it))]).collect())

from pyspark import TaskContext
print(logs.map(lambda l: TaskContext.get().partitionId()).distinct().collect())
```

```
4
[(0, 1501), (1, 1500), (2, 1501), (3, 1498)]     <- (chi so partition, so dong)
```

```
[0, 1, 2, 3]                                    <- ID partition ma cac task da xu ly
```

`TaskContext.get()` chỉ gọi được *bên trong* một task; `partitionId()` cho biết task đó đang xử lý partition nào. File được chia theo **byte**, không theo dòng, nên số dòng mỗi partition xấp xỉ nhau chứ không bằng nhau tuyệt đối. Log INFO của Spark (Câu 17) ghi rõ từng task gắn với partition nào:

```
INFO TaskSetManager: Starting task 0.0 in stage 100.0 (TID 296) (192.168.1.8, executor driver, partition 0,
     PROCESS_LOCAL, 8842 bytes)
INFO TaskSetManager: Starting task 1.0 in stage 100.0 (TID 297) (192.168.1.8, executor driver, partition 1,
     PROCESS_LOCAL, 8842 bytes)
INFO HadoopRDD: Input split: file:/.../access.log:0+62969
INFO HadoopRDD: Input split: file:/.../access.log:62969+62969
```

9.3 Bảng đo: số task trong các tình huống khác nhau

Số liệu dưới đây đọc từ REST API của Spark UI, với `logs` có 4 partition.

Lệnh	Job → stage (số task)	Giải thích
`logs.count()`	1 job → 1 stage (**4**)	một stage, 4 partition → 4 task
`logs.take(1)`	1 job → 1 stage (**1**)	chỉ cần partition đầu tiên
`logs.take(5000)`	2 job → (1), rồi (3)	partition 0 chưa đủ 5000 dòng → Spark chạy thêm job cho 3 partition còn lại
`pairs.reduceByKey(add).collect()`	1 job → (4) + (4)	2 stage; sau shuffle giữ 4 partition
`pairs.reduceByKey(add, numPartitions=3)`	1 job → (4) + (**3**)	số task stage sau = `numPartitions`
`logs.coalesce(2).count()`	1 job → 1 stage (**2**)	2 partition mới, mỗi cái gộp 2 partition cũ
`agg.collect()` (lần 2, cùng RDD)	1 job → (4, **skipped**) + (4)	kết quả shuffle lần trước còn trên đĩa

9.4 Bốn chi tiết tránh hiểu sai

1. **"Partition của stage" là partition của RDD cuối trong stage.** Sau `coalesce(2)`, stage có 2 task. Mỗi task vẫn ứng với *một* partition (của RDD đã coalesce), nhưng partition đó được tạo ra bằng cách *đọc 2 partition cha*. Câu "mỗi task đọc đúng một partition file" khi đó không còn đúng.
2. **Action có thể chỉ chạy một phần.** `take(n)`, `first()` chạy task cho số partition tối thiểu cần thiết, tăng dần nếu chưa đủ. Vì vậy `take` rất nhanh trên dữ liệu lớn.
3. **Stage bị bỏ qua (skipped).** Kết quả shuffle được giữ trên đĩa; job sau dùng lại RDD đã shuffle thì stage phía trước không chạy lại — Spark UI hiện chữ *skipped*, không có task nào.
4. **Task và lần chạy (attempt).** Log ghi `task 0.0`: số trước dấu chấm là chỉ số task (= partition), số sau là *lần thử*. Nếu task lỗi hoặc chạy quá chậm (speculative execution), Spark chạy lại thành `task 0.1` — vẫn là cùng một task, cùng partition. `TID` là số định danh duy nhất của mỗi lần thử.

Hệ quả thực hành

- Số task *chạy đồng thời* = min(số task, số core). 4 task trên máy 8 luồng → 4 luồng nhàn rỗi.

- Muốn thay đổi mức song song của một stage, hãy thay đổi **số partition** (`minPartitions`, `numPartitions`, `repartition`, `coalesce`) — không có tham số "số task" riêng.
- Một partition quá lớn → một task quá lâu → cả stage phải đợi (skew) [slide 3.14].

Câu 10. Mối quan hệ giữa task và executor

Trả lời ngắn

Executor là *nơi làm việc*: một tiến trình JVM sống suốt thời gian của Application, có một số **core (slot)** và bộ nhớ. **Task** là *đầu việc*: tính một partition của một stage, sống vài mili giây đến vài giây. Quan hệ là **nhiều–một**: mỗi task (mỗi lần chạy) chạy trên **đúng một** executor; mỗi executor chạy **rất nhiều** task trong đời nó, **đồng thời tối đa bằng số slot**. Task nhiều hơn tổng số slot thì phải xếp hàng, chạy thành **từng đợt**.

10.1 Hai khái niệm ở hai quy mô khác nhau

	Executor	Task
Bản chất	tiến trình JVM (tài nguyên)	đơn vị công việc (stage × partition)
Thuộc về	một Application (không dùng chung)	một stage của một job
Tạo bởi	cluster manager, khi Application khởi động	DAGScheduler, khi action chạy
Sống bao lâu	suốt Application (phút, giờ)	mili giây đến giây
Số lượng	vài đến vài trăm	hàng nghìn đến hàng triệu
Chứa gì	slot (core), bộ nhớ, cache, file shuffle, Python worker	hàm cần chạy (closure) + partition cần tính
Khi hỏng	mọi task đang chạy trên nó được chạy lại nơi khác	được chạy lại (attempt mới), có thể trên executor khác

Slot

Mỗi executor có `spark.executor.cores` core; mỗi task cần `spark.task.cpus` core (mặc định 1). Số task một executor chạy *cùng lúc* = số **slot** = `executor.cores / task.cpus`. Tổng slot của Application = số executor × slot mỗi executor. Với PySpark, mỗi slot đang bận dùng thêm một **Python worker** để chạy hàm Python.

10.2 Thí nghiệm: 2 executor thật, task chạy theo đợt

Chế độ `local[*]` chỉ có một executor (`driver`), khó thấy quan hệ nhiều–một. Spark có chế độ `local-cluster[N,C,M]` mô phỏng một cluster thật ngay trên máy: **N executor riêng biệt**, mỗi executor C core, M MB bộ nhớ. Ta chạy 2 executor × 2 core = **4 slot**, và cho mỗi task ngủ 1 giây để dễ đo:

```
spark = (SparkSession.builder.master("local-cluster[2,2,1024]")   # 2 executor x 2 core x 1 GB
         .appName("Q16-Task-Executor").getOrCreate())
sc = spark.sparkContext

def cham(i):
    time.sleep(1)                                  # moi task mat dung 1 giay
```

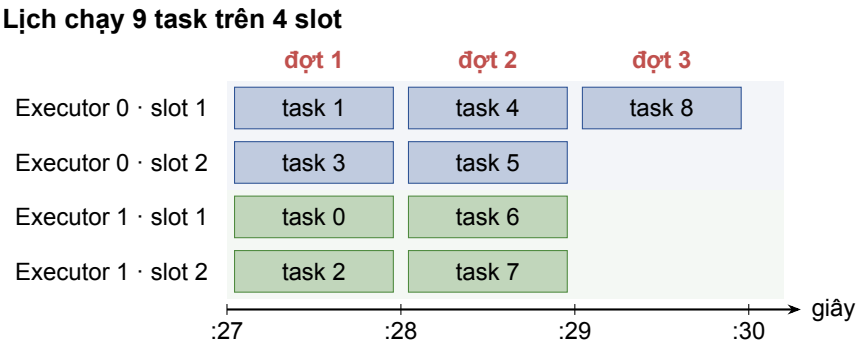
```
    tc = TaskContext.get()
    return (tc.partitionId(), os.getpid())

for n in (4, 5, 8, 9):                                  # n partition -> n task
    t0 = time.perf_counter()
    out = sc.parallelize(range(n), n).map(cham).collect()
    print(f"{n} task: {time.perf_counter() - t0:.1f} s | so Python worker: {len(set(o[1] for o
     in out))}")
```

```
executors: [('driver', 'macbookpro:50538', 0), ('1', '192.168.79.126:50546', 2), ('0',
    '192.168.79.126:50547', 2)]
4 task: 1.1 s | so Python worker: 4          <- 4 task <= 4 slot: 1 dot
5 task: 2.1 s | so Python worker: 4          <- 4 + 1: 2 dot (dot 2 chi dung 1/4 slot)
8 task: 2.1 s | so Python worker: 4          <- 4 + 4: 2 dot, day du
9 task: 3.1 s | so Python worker: 4          <- 4 + 4 + 1: 3 dot
```

Chi tiết 9 task của job cuối, đọc từ REST API của Spark UI (Câu 17):

```
(index, executorId, host,            launch,    duration)
(0, '1', '192.168.79.126', '04:22:27', 1.0)
(1, '0', '192.168.79.126', '04:22:27', 1.0)
(2, '1', '192.168.79.126', '04:22:27', 1.0)
(3, '0', '192.168.79.126', '04:22:27', 1.0)
(4, '0', '192.168.79.126', '04:22:28', 1.0)
(5, '0', '192.168.79.126', '04:22:28', 1.0)
(6, '1', '192.168.79.126', '04:22:28', 1.0)
(7, '1', '192.168.79.126', '04:22:28', 1.0)
(8, '0', '192.168.79.126', '04:22:29', 1.0)
tong task moi executor (ca ung dung): [('driver', 0), ('1', 15), ('0', 19)]
```



Thời điểm và executor của từng task là số liệu thật; việc task nằm ở slot 1 hay slot 2 *bên trong* một executor là minh hoạ (Spark không ghi lại số slot).

10.3 Rút ra từ thí nghiệm

1. **Mỗi task chạy trên đúng một executor.** Cột *executorId* của mỗi task chỉ có một giá trị.
2. **Một executor chạy nhiều task**: vừa *đồng thời* (2 task cùng lúc = 2 slot), vừa *nối tiếp* (task 4, 5 chạy sau task 1, 3 trên Executor 0). Cả ứng dụng, Executor 0 chạy 19 task, Executor 1 chạy 15 task.
3. **Driver lập lịch, executor thực thi.** Executor `driver` có 0 core — trong chế độ cluster, Driver không chạy task nào. Bộ lập lịch (TaskScheduler) trên Driver giao task cho slot trống, ưu tiên executor đang giữ sẵn dữ liệu (*data locality*: `PROCESS_LOCAL` → `NODE_LOCAL` → `RACK_LOCAL` → `ANY`).

4. **Thời gian ≈ số đợt × thời gian một task**, với số đợt $= \lceil$số task/tổng slot$\rceil$. 5 task mất gần bằng 8 task: đợt thứ hai chỉ dùng 1/4 máy.
5. **4 Python worker** cho 4 slot: mỗi slot đang bận có một tiến trình Python riêng, được dùng lại qua các đợt.

Hệ quả thực hành

- Chọn số partition là **bội số của tổng slot** để đợt cuối không lãng phí; kết hợp với quy tắc 2–4 × số core [slide 3.7]: nhiều đợt nhỏ giúp cân bằng tải khi task dài ngắn khác nhau.
- Một task quá chậm (skew) giữ một slot; các slot khác rảnh chờ — stage kết thúc khi task *cuối cùng* xong.
- Trên cluster, bộ nhớ của **executor** phải đủ cho *tất cả* task chạy đồng thời trên nó cộng với cache.

Câu 11. Khái niệm parallelize và serialize

Trả lời ngắn

Hai khái niệm khác nhau nhưng luôn đi cùng nhau trong Spark. **Parallelize** (song song hoá) là *chia* dữ liệu/công việc thành nhiều phần để xử lý **đồng thời**; trong PySpark, `sc.parallelize(list)` biến một list trên Driver thành RDD có nhiều partition. **Serialize** (tuần tự hoá) là *chuyển một đối tượng thành chuỗi byte* để gửi qua mạng, ghi đĩa hoặc chuyển giữa hai tiến trình; **deserialize** là làm ngược lại. Muốn song song hoá trên nhiều máy thì dữ liệu và hàm phải được serialize để gửi đi — nên mọi lần ”song song” trong Spark đều kèm chi phí ”tuần tự hoá”.

11.1 Song song hoá nói chung

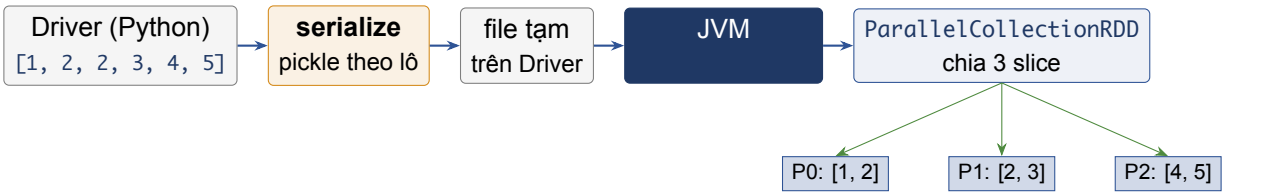
Định nghĩa

Song song hoá là tổ chức một công việc sao cho nhiều phần của nó được thực hiện **cùng lúc** trên nhiều bộ xử lý.

- **Song song dữ liệu** (data parallelism): *cùng một phép toán* áp dụng lên *các phần khác nhau* của dữ liệu — đây là mô hình của Spark (mỗi task chạy cùng một hàm trên một partition).
- **Song song tác vụ** (task parallelism): *các phép toán khác nhau* chạy đồng thời.

Giới hạn: phần công việc *không* chia được (đọc cấu hình, gộp kết quả cuối, chờ task chậm nhất) quyết định tốc độ tối đa — đó là lý do shuffle và skew quan trọng. Trên máy này, mức song song tối đa là `sc.defaultParallelism = 8`.

11.2 `sc.parallelize`: từ list trên Driver thành RDD



task *deserialize* slice của mình trong Python worker

```
nums = sc.parallelize([1, 2, 2, 3, 4, 5], 3)
print(nums.glom().collect())
print(nums.toDebugString().decode())
print(nums._jrdd_deserializer)                    # thuoc tinh noi bo, chi de tim hieu
print(sc.textFile(f"{DATA}/access.log")._jrdd_deserializer)
print(sc.parallelize(range(10), 3).glom().collect())
```

```
[[1, 2], [2, 3], [4, 5]]
(3) ParallelCollectionRDD[0] at readRDDFromFile at PythonRDD.scala:287
BatchedSerializer(PickleSerializer(), 2)       <- list duoc pickle theo lo, 2 phan tu/lo
UTF8Deserializer(True)                         <- textFile: moi dong la chuoi UTF-8, khong can
    pickle
[[0, 1, 2], [3, 4, 5], [6, 7, 8, 9]]
```

List và range khác nhau thế nào? Với `list`, toàn bộ phần tử phải được pickle rồi ghi ra file tạm trên Driver. Với `range`, PySpark chỉ gửi *điểm đầu, bước, điểm cuối* của từng slice — mỗi task tự sinh số của mình:

```
r1 = sc.parallelize(list(range(2_000_000)), 8)     # tao RDD: 0.48 s ; count: 0.18 s
r2 = sc.parallelize(range(2_000_000), 8)           # tao RDD: 0.0034 s ; count: 0.11 s
```

Lưu ý

`parallelize` làm dữ liệu **đi qua Driver** (serialize trên một máy). Nó chỉ dùng cho dữ liệu nhỏ, dữ liệu thử nghiệm, hoặc danh sách tham số. Dữ liệu lớn phải đọc bằng `textFile`/`spark.read` để mỗi executor tự đọc phần của mình.

11.3 Serialize và deserialize

Định nghĩa

Serialize (tuần tự hoá, *marshalling*): chuyển một đối tượng trong bộ nhớ (tuple, dict, hàm…) thành một **chuỗi byte** độc lập với tiến trình, để lưu hoặc truyền đi. **Deserialize**: dựng lại đối tượng từ chuỗi byte ở phía nhận. Python dùng `pickle`; PySpark dùng thêm `cloudpickle` cho hàm; phía JVM dùng Java serializer hoặc Kryo.

```
import pickle
rec = ("ERROR", "10.0.3.185", "POST", "/api/search", 502, 358)
b = pickle.dumps(rec)
print(type(b).__name__, len(b), "byte |", b[:24])
print(pickle.loads(b) == rec)
```

```
bytes 71 byte | b'\x80\x03(X\x05\x00\x00\x00ERRORq\x00X\n\x00\x00\x0010.0'
True
```

Bản ghi dạng chuỗi gốc chỉ 43 byte; sau khi parse thành tuple rồi pickle là 71 byte — serialize có chi phí cả **thời gian** lẫn **dung lượng**. Với 1 triệu bản ghi khác nhau: `dumps` 0,63 s, `loads` 0,44 s, 27,3 MB.

Hàm cũng phải được serialize — pickle không làm được, cloudpickle thì được

```
nguong = 3
pickle.dumps(lambda x: x > nguong)                # pickle thuong
from pyspark import cloudpickle
b = cloudpickle.dumps(lambda x: x > nguong)       # PySpark dung cai nay cho closure
print(len(b), cloudpickle.loads(b)(5))
```

```
PicklingError: Can't pickle <function <lambda> ...>: attribute lookup <lambda> on __main__
    failed
500 True        <- cloudpickle dong goi ca ma ham lan bien nguong (Cau 14)
```

Những gì không serialize được

Kiểm chứng bằng `cloudpickle.dumps(lambda x: ...)` tham chiếu tới từng đối tượng:

Đối tượng trong closure	Kết quả
`threading.Lock()`	`TypeError: can't pickle _thread.lock objects`
`socket.socket()`	`TypeError: Cannot serialize socket object`
`sqlite3.connect(...)`	`TypeError: can't pickle sqlite3.Connection objects`
file mở để **ghi**	`PicklingError: Cannot pickle files that are not opened for reading: w`
file mở để **đọc**	**không báo lỗi** — cloudpickle **chép toàn bộ nội dung file** (252.540 byte với `access.log` 251.876 byte) vào closure, gửi kèm mỗi task

Cách đúng: tạo tài nguyên (kết nối, file) **bên trong** `mapPartitions`/`foreachPartition`, trên executor, mỗi partition một lần.

11.4 Spark serialize ở những đâu?

Thời điểm	Chiều	Cái gì được serialize
`parallelize`	Driver → JVM → executor	phần tử của list (pickle theo lô)
Gửi task	Driver → executor	closure: hàm + biến (cloudpickle) — Câu 14
Chạy hàm Python	JVM ↔ Python worker	từng lô phần tử, qua socket (pickle) — Câu 2
Shuffle	executor → đĩa → mạng → executor	bản ghi trung gian (kèm nén)
`cache()`	trong bộ nhớ executor	phần tử đã pickle — *Memory Serialized* (Câu 18)
`collect`, `take`…	executor → Driver	kết quả của action
`broadcast`	Driver → mỗi executor	giá trị broadcast (một lần) — Câu 14

Hiệu năng

Với RDD Python, serialize/deserialize thường là chi phí **lớn nhất** sau shuffle: mỗi phần tử đi Python → byte → JVM → byte → Python. Đây là lý do: (1) lọc sớm, bớt cột giúp giảm lượng phải serialize; (2) cache dạng serialize trong PySpark không nhanh hơn bao nhiêu khi bước phía trước rẻ [slide 5.7]; (3) DataFrame nhanh hơn vì dữ liệu ở lại trong JVM, dạng nhị phân

gọn, không qua Python.

	Parallelize	**Serialize**
Trả lời câu hỏi	chia việc/dữ liệu thế nào để chạy đồng thời?	đóng gói đối tượng thế nào để gửi/lưu?
Kết quả	nhiều partition, nhiều task	chuỗi byte
Trong PySpark	`sc.parallelize`, số partition, `local[*]`	`pickle`, `cloudpickle`, `BatchedSerializer`
Quan hệ	song song hoá trên nhiều tiến trình/máy **cần** serialize để phân phát dữ liệu và hàm	

PHẦN IV

Phép toán và mô hình thực thi

Câu 12 – 15 · slide 3.10 – 3.21: transformation/action, narrow/wide, closure, bảng tra cứu

Câu 12. `reduceByKey` là transformation, `reduce` là action; `sample` là transformation, `takeSample` là action

Trả lời ngắn

Tiêu chí duy nhất để phân loại là **kiểu giá trị trả về**. `reduceByKey` và `sample` trả về **RDD** — kết quả vẫn phân tán, chưa được tính → transformation, lazy. `reduce` và `takeSample` trả về **giá trị Python** (số, list) nằm trên Driver → phải tính ngay → action. Lý do thiết kế: kết quả của `reduceByKey` có thể rất lớn (một dòng cho mỗi khoá, có thể hàng triệu khoá) nên phải để phân tán; kết quả của `reduce` chỉ là **một giá trị** nên đưa về Driver an toàn.

12.1 `reduceByKey` và `reduce`

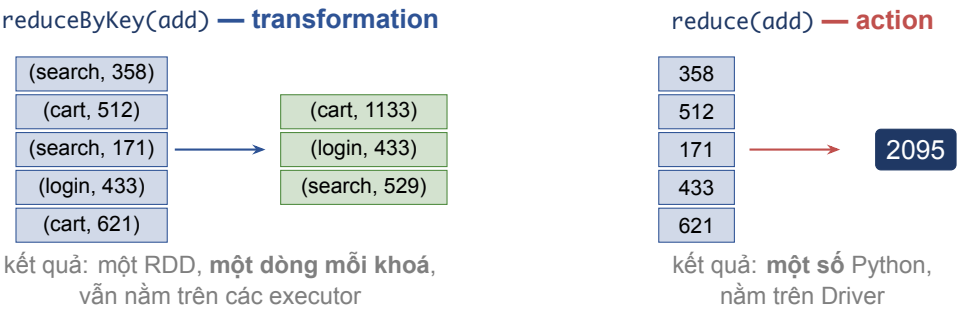
```
kv = sc.parallelize([("search", 358), ("cart", 512), ("search", 171), ("login", 433), ("cart",
     621)], 2)

sc.setJobGroup("q7_a", "q7_a")
x = kv.reduceByKey(add)                          # transformation
print(type(x).__name__, "| so job da chay:", sc.statusTracker().getJobIdsForGroup("q7_a"))

y = kv.map(lambda t: t[1]).reduce(add)           # action
print(type(y).__name__, y, "| so job da chay:", sc.statusTracker().getJobIdsForGroup("q7_a"))

print(sorted(x.collect()))                       # can mot action de thay ket qua cua x
```

```
PipelinedRDD | so job da chay: []            <- tra ve tuc thi (0,017 s), KHONG co job nao
int 2095 | so job da chay: [33]              <- chay ngay 1 job, tra ve so nguyen
[('cart', 1133), ('login', 433), ('search', 529)]
```



	`reduceByKey(f)`	`reduce(f)`
Đầu vào	pair RDD `(k, v)`	RDD bất kỳ
Gộp theo	**từng khoá**	**toàn bộ** RDD
Trả về	RDD `(k, v)` — một dòng mỗi khoá	một giá trị Python
Kích thước kết quả	có thể rất lớn (bằng số khoá)	luôn là một giá trị
Kết quả nằm ở	các executor (phân tán)	Driver
Khi gọi	không chạy gì (lazy)	chạy job ngay
Shuffle	có (wide)	không (gộp cục bộ rồi gửi về Driver)
Việc tiếp theo	nối thêm transformation, rồi một action	dùng như biến Python bình thường

Lỗi hay gặp

- Viết `counts = pairs.reduceByKey(add)` rồi dừng — không có gì được tính. Cần `collect()`, `take()`, `saveAsTextFile()`…
- Coi `reduceByKey` như trả về `dict`: `counts["search"]` báo lỗi, vì `counts` là RDD. Muốn dict nhỏ trên Driver: `dict(counts.collect())` — chỉ khi số khoá nhỏ.
- Cả hai đều yêu cầu hàm `f` có **tính kết hợp và giao hoán** [slide 4.22–4.24].

12.2 `sample` và `takeSample`

```
big = sc.parallelize(range(1000), 4)

sc.setJobGroup("q7_s", "q7_s")
s = big.sample(False, 0.1, seed=42)                    # transformation
print(type(s).__name__, "| jobs:", sc.statusTracker().getJobIdsForGroup("q7_s"))
print([big.sample(False, 0.1, seed=k).count() for k in (7, 42, 2024, 99)])

sc.setJobGroup("q7_t", "q7_t")
ts = big.takeSample(False, 5, seed=42)                 # action
print(type(ts).__name__, ts, "| so job:", len(sc.statusTracker().getJobIdsForGroup("q7_t")))
print({len(big.takeSample(False, 100, seed=k)) for k in range(3)})
```

```
PipelinedRDD | jobs: []            <- chua chay gi
[85, 98, 86, 91]                  <- fraction=0.1 la XAC SUAT: so luong dao dong quanh 100
list [665, 655, 91, 646, 319] | so job: 2
{100}                             <- takeSample luon tra DUNG so phan tu yeu cau
```

	`sample(withReplacement, fraction, seed)`	`takeSample(withReplacement, num, seed)`
Tham số kích thước	`fraction`: **xác suất** chọn mỗi phần tử	`num`: **số lượng chính xác**
Số phần tử kết quả	dao động quanh `fraction` × N	đúng `num` (nếu RDD đủ lớn)
Trả về	RDD — mẫu vẫn phân tán	`list` Python trên Driver
Khi gọi	lazy, không chạy gì	chạy ngay: 1 job đếm + 1 job lấy mẫu (có thể thêm nếu thiếu)
Chi phí	rẻ (narrow), chạy trong cùng stage	duyệt toàn bộ RDD, kết quả về Driver
Dùng khi	cần một **tập con lớn** để xử lý tiếp: chạy thử pipeline trên 1% dữ liệu	cần **vài phần tử** để xem, in ra, vẽ

12.3 Quy tắc nhận biết chung và các cặp dễ nhầm

Quy tắc

Hỏi: *"Kết quả của lệnh này còn là RDD không?"* Còn → transformation (lazy, phân tán). Không (số, list, dict, file, không gì cả) → action (chạy ngay, kết quả về Driver hoặc ra ngoài).

Transformation	Action	Khác nhau ở đâu
`reduceByKey(f)`	`reduce(f)`	theo khoá, ra RDD / toàn bộ, ra một giá trị
`sample(...)`	`takeSample(...)`	xác suất, ra RDD / số lượng chính xác, ra list
`foldByKey(z, f)`	`fold(z, f)`	như cặp `reduce`, có giá trị khởi tạo
`aggregateByKey(...)`	`aggregate(...)`	như trên, cho phép đổi kiểu kết quả
`map(f)`	`foreach(f)`	tạo RDD mới / chỉ chạy `f` vì hiệu ứng phụ
`sortBy(f)`	`takeOrdered(n, key)`, `top(n)`	sắp xếp cả RDD (wide) / chỉ lấy n phần tử nhỏ/lớn nhất
`reduceByKey(add)`	`countByKey()`	**bẫy**: `countByKey` có chữ "ByKey" nhưng là **action**, trả `dict` về Driver

Câu 13. Wide–long table trong pandas và narrow–wide transformation trong PySpark

Trả lời ngắn

Đây là **hai khái niệm khác nhau, chỉ trùng chữ "wide"**. Wide/long table nói về *hình dạng bảng* (dữ liệu trải theo cột hay theo dòng). Narrow/wide transformation nói về *phép toán có cần shuffle hay không*. Mối liên hệ nằm ở chỗ khác: khi làm trên Spark, **chuyển wide → long** (`melt`) là phép **narrow**, còn **chuyển long → wide** (`pivot`) là phép **wide**, vì nó phải gom các dòng cùng khoá nằm rải rác.

13.1 Wide table và long table

Định nghĩa

- **Wide table** (bảng rộng): mỗi **đối tượng** là một dòng, mỗi **biến hoặc thời điểm** là một cột riêng.
- **Long table** (bảng dài, còn gọi là *tidy data*): mỗi **quan sát** là một dòng; tên biến được đưa vào **một cột**, giá trị vào **một cột** khác.

Hai bảng chứa **cùng thông tin**; chỉ khác cách sắp xếp.

```
import pandas as pd
wide = pd.DataFrame({"store": ["A", "B"], "T1": [100, 80], "T2": [120, 95], "T3": [90, 110]})
long = wide.melt(id_vars="store", var_name="month", value_name="revenue")   # wide -> long
back = long.pivot(index="store", columns="month", values="revenue")         # long -> wide
```

wide

	store	T1	T2	T3
0	A	100	120	90
1	B	80	95	110

long

	store	month	revenue
0	A	T1	100
1	B	T1	80
2	A	T2	120
3	B	T2	95
4	A	T3	90
5	B	T3	110

back (lại là wide)

month	T1	T2	T3
store			
A	100	120	90
B	80	95	110

Khi nào dùng dạng nào?

	Wide table	Long table
Người đọc chính	Con người: báo cáo, Excel, so sánh theo hàng	Máy: lọc, gom nhóm, vẽ biểu đồ
Gom nhóm, tính toán	Khó: muốn tính trung bình theo tháng phải liệt kê từng cột	Dễ: `long.groupby("month")["revenue"].mean()`
Thêm kỳ mới (tháng T4)	Phải **thêm cột** — schema thay đổi	Chỉ **thêm dòng** — schema giữ nguyên
Vẽ biểu đồ (seaborn, plotly)	Thường phải chuyển đổi trước	Dùng trực tiếp: `x="month", hue="store"`
Học máy	Đúng dạng ma trận đặc trưng X	Phải pivot trước
Lưu trữ dữ liệu lớn	Schema hay đổi, nhiều ô rỗng	Schema ổn định, dễ nối thêm

```
long.groupby("month")["revenue"].mean()
```

```
month
T1     90.0
T2    107.5
T3    100.0
```

Quy tắc thực hành: **lưu trữ và xử lý ở dạng long**; chỉ **pivot sang wide ở bước cuối**, khi cần trình bày hoặc đưa vào mô hình học máy.

13.2 Nhắc lại: narrow và wide transformation

Định nghĩa

- **Narrow**: mỗi partition của RDD cha được dùng bởi **nhiều nhất một** partition của RDD con. Không di chuyển dữ liệu.
- **Wide**: một partition của RDD cha được dùng bởi **nhiều** partition của RDD con → phải **shuffle**.

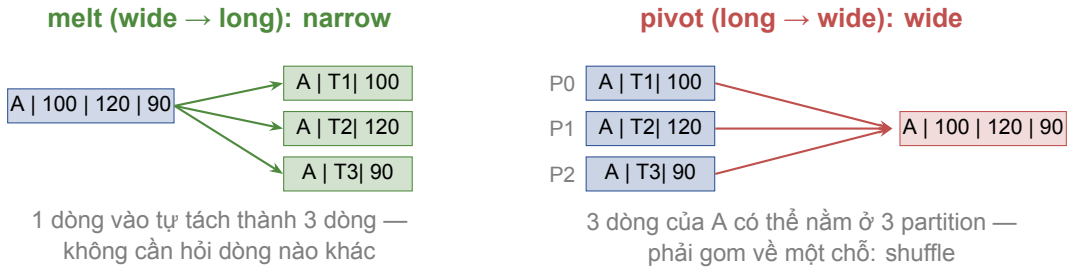
	Wide / long table	**Narrow / wide transformation**
Nói về	Hình dạng của bảng	Cách dữ liệu di chuyển giữa các partition
"Wide" nghĩa là	Bảng có nhiều cột	Phép toán cần shuffle
Cặp đối lập	wide ↔ **long**	wide ↔ **narrow**
Bối cảnh	pandas, R, thiết kế dữ liệu nói chung	Spark, xử lý phân tán

Hiểu sai thường gặp

"Bảng wide thì phải xử lý bằng wide transformation" là **sai**. Một bảng wide có thể được xử lý hoàn toàn bằng phép narrow (ví dụ `map` tính tổng ba cột trên từng dòng), và một bảng long vẫn có thể cần shuffle (ví dụ gom nhóm).

13.3 Mối liên hệ thật sự: phép chuyển đổi giữa hai dạng bảng

Hãy tự hỏi câu hỏi cốt lõi của Spark: *để tạo ra một dòng kết quả, cần những dòng đầu vào nào?*



Kiểm chứng bằng RDD

```
wide_rdd = sc.parallelize([("A", 100, 120, 90), ("B", 80, 95, 110)], 2)
MONTHS = ["T1", "T2", "T3"]

# wide -> long: flatMap (moi dong tu tach ra)
long_rdd = wide_rdd.flatMap(lambda r: [(r[0], m, v) for m, v in zip(MONTHS, r[1:])])
print(long_rdd.collect())
print(long_rdd.toDebugString().decode())
```

```
[('A', 'T1', 100), ('A', 'T2', 120), ('A', 'T3', 90), ('B', 'T1', 80), ('B', 'T2', 95), ('B',
    'T3', 110)]
(2) PythonRDD[1] at collect at ...
 |  ParallelCollectionRDD[0] at readRDDFromFile at PythonRDD.scala:287
```

```
# long -> wide: phai gom cac dong cung store -> groupByKey
wide_again = (long_rdd.map(lambda t: (t[0], (t[1], t[2])))
                      .groupByKey()
                      .mapValues(lambda vs: dict(sorted(vs))))
print(sorted(wide_again.collect()))
print(wide_again.toDebugString().decode())
```

```
[('A', {'T1': 100, 'T2': 120, 'T3': 90}), ('B', {'T1': 80, 'T2': 95, 'T3': 110})]
(2) PythonRDD[6] at collect at ...
 |  MapPartitionsRDD[5] at mapPartitions at PythonRDD.scala:158
 |  ShuffledRDD[4] at partitionBy at NativeMethodAccessorImpl.java:0      <- SHUFFLE
 +-(2) PairwiseRDD[3] at groupByKey at ...
    |  PythonRDD[2] at groupByKey at ...
    |  ParallelCollectionRDD[0] at readRDDFromFile at PythonRDD.scala:287
```

Lineage của `melt` không có `ShuffledRDD` (một stage); lineage của `pivot` có `ShuffledRDD` và dòng `+-` thụt vào — ranh giới stage.

Kiểm chứng bằng Spark DataFrame

Với DataFrame, ta đọc kế hoạch vật lý bằng `explain()`; mỗi lần shuffle hiện ra là một nút `Exchange`.

```
df_long.groupBy("store").pivot("month").sum("revenue").explain()                    # long ->
    wide
df_wide.selectExpr("store", "stack(3, 'T1', T1, 'T2', T2, 'T3', T3) as (month, revenue)").
    explain()  # wide -> long
```

```
== pivot ==
... HashAggregate(keys=[store#0], functions=[pivotfirst(...)])
      +- Exchange hashpartitioning(store#0, 200)                  <- shuffle 1
         +- HashAggregate(keys=[store#0], functions=[partial_pivotfirst(...)])
            +- HashAggregate(keys=[store#0, month#1], functions=[sum(revenue#2L)])
               +- Exchange hashpartitioning(store#0, month#1, 200)   <- shuffle 2
                  ...
== stack ==
*(1) Generate stack(3, T1, T1#119L, T2, T2#120L, T3, T3#121L), [store#118], ...
+- *(1) Scan ExistingRDD[store#118,T1#119L,T2#120L,T3#121L]       <- khong co Exchange
```

Hiệu năng

- `pivot` trong Spark SQL dùng tới **hai** lần shuffle (gộp theo `(store, month)` rồi theo `store`). Nếu không chỉ định danh sách giá trị, Spark còn chạy thêm một job để tìm các giá trị khác nhau của cột `month`. Nên viết `pivot("month", ["T1", "T2", "T3"])` khi đã biết trước các giá trị.
- `melt`/`stack` là narrow, chạy trong cùng stage với các bước trước nó — gần như "miễn phí".
- Vì vậy: **giữ dữ liệu ở dạng long trong suốt pipeline**, chỉ pivot ở cuối, trên dữ liệu đã được lọc và gộp nhỏ lại.

Câu 14. Closure và biến chia sẻ: Accumulator, Broadcast [slide 3.19]

Trả lời ngắn

Hàm bạn truyền vào `map`/`filter`/`foreach` **không chạy trên Driver**: Spark đóng gói hàm *cùng mọi biến nó dùng* (gọi là **closure**), gửi **bản sao** tới từng task, và chạy trong các Python worker riêng. Hệ quả: executor **đọc** được biến của Driver, nhưng **sửa** biến đó thì Driver không hề biết. Spark cung cấp hai loại biến chia sẻ cho hai chiều ngược nhau: **Accumulator** để executor *gửi về* Driver (chỉ cộng dồn), và **Broadcast** để Driver *gửi đi* một giá trị lớn chỉ đọc, hiệu quả hơn closure.

14.1 Closure trong Python

Định nghĩa

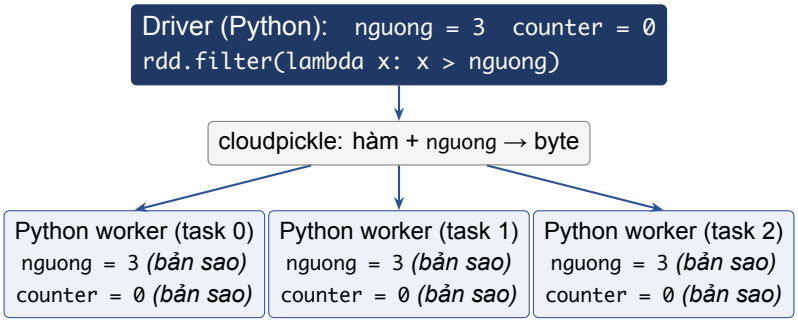
Closure là một hàm *cùng với các biến bên ngoài* mà nó tham chiếu tới (biến tự do). Hàm "mang theo" môi trường nơi nó được định nghĩa.

```
def tao_bo_loc(nguong):
    def loc(x):
        return x > nguong        # 'nguong' la bien tu do -> duoc "dong goi" cung ham loc
    return loc

lon_hon_3 = tao_bo_loc(3)
print(lon_hon_3(5), lon_hon_3.__closure__[0].cell_contents)
```

```
True 3
```

14.2 Closure trong Spark: gửi bản sao tới từng task



`counter += 1` chỉ sửa bản sao trong worker — không có đường nào quay về Driver

Ba sự thật cần nhớ, đều đã kiểm chứng:

(1) Sửa biến trong hàm không làm thay đổi biến trên Driver — kể cả ở chế độ local

```
counter = 0
def inc(x):
    global counter
    counter += 1
nums.foreach(inc)                          # nums co 6 phan tu
print("counter tren Driver:", counter)

seen = []
```

```
nums.foreach(lambda x: seen.append(x))
print("list tren Driver:", seen)
```

```
counter tren Driver: 0
list tren Driver: []
```

Nhiều sinh viên nghĩ "chạy `local[*]` trên cùng máy thì biến là một". Không đúng: hàm chạy trong các tiến trình `pyspark.daemon` riêng (Câu 2), có bộ nhớ riêng.

(2) Giá trị biến được "chụp" khi action đầu tiên chạy, rồi cố định

```
nguong = 3
r = nums.filter(lambda x: x > nguong)     # dinh nghia khi nguong = 3
nguong = 4                                # doi TRUOC action dau tien
print("action 1:", r.collect())
nguong = 0                                # doi SAU action dau tien
print("action 2 (cung RDD r):", r.collect())
print("RDD moi:", nums.filter(lambda x: x > nguong).collect())
```

```
action 1: [5]                <- dung nguong = 4 (gia tri luc action chay, khong phai luc dinh
     nghia)
action 2 (cung RDD r): [5]   <- van nguong = 4: ham da duoc dong goi va luu lai
RDD moi: [1, 2, 2, 3, 4, 5]  <- RDD moi dong goi lai voi nguong = 0
```

Trong PySpark, hàm được serialize khi RDD được gửi sang JVM lần đầu (ở action đầu tiên) và kết quả được giữ lại (`_jrdd_val`). Bài học: **không thay đổi biến mà closure dùng** giữa lúc định nghĩa và lúc chạy — code sẽ khó đoán.

(3) Mọi biến tham chiếu đều bị gửi kèm — kể cả biến rất lớn

```
from pyspark import cloudpickle
big_lookup = {f"k{i}": i for i in range(200000)}
f_closure = lambda ip: big_lookup.get(ip)          # tham chieu truc tiep
bc = sc.broadcast(big_lookup)
f_bc      = lambda ip: bc.value.get(ip)            # tham chieu qua broadcast
print(len(cloudpickle.dumps(f_closure)), len(cloudpickle.dumps(f_bc)))
```

```
2758851 550        <- 2,76 MB gui kem MOI TASK, so voi 550 byte
```

Với 1000 task, cách thứ nhất gửi khoảng 2,7 GB qua mạng; cách thứ hai chỉ gửi khoảng 550 KB (550 byte × 1000 task), còn bảng tra cứu được gửi **một lần cho mỗi executor**.

14.3 Accumulator — kênh một chiều từ executor về Driver

Định nghĩa

Accumulator là biến chia sẻ **chỉ cộng thêm**. Task gọi `acc.add(v)` (hoặc `acc += v`) trên bản cục bộ; Spark gửi các phần cộng về Driver và gộp lại. Chỉ Driver đọc được `acc.value`.

```
acc = sc.accumulator(0)
nums.foreach(lambda x: acc.add(1))                # trong ACTION
print("accumulator trong action:", acc.value)

acc2 = sc.accumulator(0)
def dem(x):
```

```
    acc2.add(1)
    return x
m = nums.map(dem)                                  # trong TRANSFORMATION
m.count(); m.count()                               # 2 action, khong cache
print("accumulator trong transformation, 2 action:", acc2.value)
```

```
accumulator trong action: 6
accumulator trong transformation, 2 action: 12     <- dem 2 lan vi map chay lai
```

Đảm bảo của Accumulator

- Cập nhật trong **action** (`foreach`): Spark đảm bảo mỗi task chỉ được tính **một lần**, kể cả khi task chạy lại.
- Cập nhật trong **transformation** (`map`): có thể bị cộng **nhiều lần** khi RDD được tính lại (nhiều action, mất cache, task chạy lại). Kết quả 12 ở trên là ví dụ. Chỉ dùng để *gỡ lỗi/đo đạc*, như ta đã dùng để chứng minh lazy evaluation [slide 4.20].
- Đọc `acc.value` bên trong task báo lỗi *"Accumulator.value cannot be accessed inside tasks"*.

Accumulator kiểu tuỳ chỉnh. Mặc định là số; muốn cộng dồn kiểu khác, định nghĩa `AccumulatorParam`:

```
from pyspark.accumulators import AccumulatorParam
class SetParam(AccumulatorParam):
    def zero(self, v):         return set()
    def addInPlace(self, a, b): a |= b; return a

eps = sc.accumulator(set(), SetParam())
logs.foreach(lambda l: eps.add({l.split()[3]}) if l.startswith("ERROR") else None)
print(sorted(eps.value))
```

```
['/api/admin', '/api/cart', '/api/checkout', '/api/login', '/api/product', '/api/search']
```

Ứng dụng

Đếm số dòng hỏng khi parse, đếm số bản ghi bị lọc bỏ, thu thập số liệu giám sát (metrics) trong khi job chạy — những thứ *phụ* bên cạnh kết quả chính.

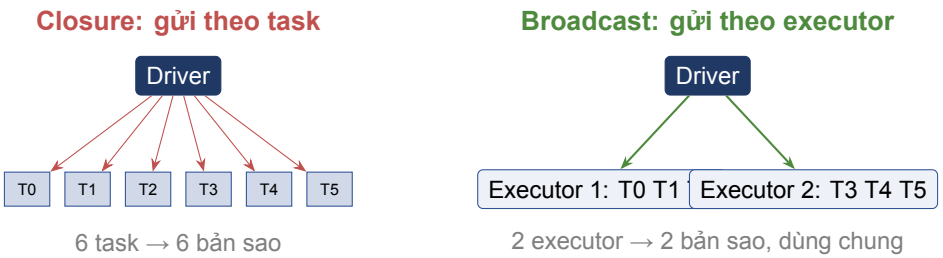
14.4 Broadcast — gửi một giá trị lớn, chỉ đọc, một lần mỗi executor

Định nghĩa

Broadcast variable là giá trị **chỉ đọc** được Driver gửi tới **mỗi executor một lần** và giữ lại để mọi task trên executor đó dùng chung. Tạo bằng `bv = sc.broadcast(v)`, đọc bằng `bv.value`, giải phóng bằng `bv.unpersist()` (có thể gửi lại) hoặc `bv.destroy()` (xoá hẳn).

```
lookup = {f"10.0.{a}.{b}": f"zone-{a % 4}" for a in range(16) for b in range(256)}
zone = sc.broadcast(lookup)
res = logs.map(lambda l: zone.value.get(l.split()[1], "unknown")).countByValue()
print(sorted(res.items()))
zone.unpersist()
```

```
[('zone-0', 1806), ('zone-1', 1769), ('zone-2', 1207), ('zone-3', 1218)]
```



Ứng dụng

Bảng tra cứu (mã → tên, IP → vùng), danh sách từ dừng (stopwords), tham số mô hình học máy, và đặc biệt **broadcast join**: ghép bảng lớn với bảng nhỏ mà không shuffle bảng lớn [slide 5.10].

Lưu ý

Broadcast là **chỉ đọc**: sửa `bv.value` trên executor không truyền đi đâu cả. Giá trị broadcast phải vừa bộ nhớ của **từng** executor (thường vài MB đến vài trăm MB).

14.5 Tổng hợp ba cơ chế

	Biến trong closure	Accumulator	Broadcast
Chiều dữ liệu	Driver → task	task → Driver	Driver → executor
Executor được	đọc; sửa bản sao (vô ích)	chỉ **cộng thêm**	chỉ **đọc**
Driver đọc kết quả	—	`acc.value`	—
Số lần gửi	mỗi task một lần	gộp khi task xong	mỗi executor một lần
Dùng cho	tham số nhỏ (ngưỡng, hằng số)	đếm, thống kê phụ	bảng tra cứu lớn, broadcast join

Câu 15. Bảng tra cứu đầy đủ: tác dụng và ví dụ của từng hàm [slide 3.21]

Trả lời ngắn

Mục này bổ sung cho slide 3.21: với **mỗi** hàm có *tác dụng*, *cú pháp*, *loại* (narrow/wide/action), *ví dụ chạy thật* và *lưu ý*. Tất cả ví dụ dùng chung bốn RDD nhỏ dưới đây, để dễ so sánh kết quả giữa các hàm.

```
from operator import add
nums  = sc.parallelize([1, 2, 2, 3, 4, 5], 2)                          # 2 partition
lines = sc.parallelize(["GET /api/search", "POST /api/cart 200", ""], 2)
kv    = sc.parallelize([("search", 358), ("cart", 512), ("search", 171),
                        ("login", 433), ("cart", 621)], 2)               # (endpoint, latency
    )
names = sc.parallelize([("search", "Tim kiem"), ("cart", "Gio hang"), ("admin", "Quan tri")])
nums.glom().collect()
```

```
[[1, 2, 2], [3, 4, 5]]
```

15.1 Transformation

`map(f)` — transformation · narrow · 1 vào → 1 ra

Tác dụng: áp dụng `f` cho từng phần tử, được RDD mới **cùng số phần tử**.

```
nums.map(lambda x: x * 10).collect()
```

```
[10, 20, 20, 30, 40, 50]
```

Dùng để parse, chuẩn hoá, tính cột mới, tạo cặp `(khoá, giá trị)`.

`filter(f)` — transformation · narrow · 1 vào → 0 hoặc 1 ra

Tác dụng: giữ phần tử mà `f(x)` trả `True`; giá trị không đổi, số lượng giảm.

```
nums.filter(lambda x: x % 2 == 0).collect()
```

```
[2, 2, 4]
```

Đặt càng sớm càng tốt trong pipeline [slide 5.3]. Số partition giữ nguyên, có thể để lại partition rỗng.

`flatMap(f)` — transformation · narrow · 1 vào → 0..n ra

Tác dụng: `f` trả về một tập (list, tuple, generator); Spark **trải phẳng** các tập thành các phần tử riêng lẻ.

```
lines.map(lambda s: s.split()).collect()        # so sanh voi map
lines.flatMap(lambda s: s.split()).collect()
```

```
[['GET', '/api/search'], ['POST', '/api/cart', '200'], []]     <- map: 3 phan tu (list)
['GET', '/api/search', 'POST', '/api/cart', '200']            <- flatMap: 5 phan tu, dong rong
     bien mat
```

Dùng cho word count, “bung” bản ghi chứa danh sách, parse chịu lỗi (trả `[]` để bỏ dòng hỏng).

`mapValues(f)` — transformation · narrow · chỉ trên pair RDD

Tác dụng: áp dụng `f` lên **giá trị**, giữ nguyên khoá — và giữ partitioner (Câu 8).

```
kv.mapValues(lambda ms: round(ms / 1000, 3)).collect()          # ms -> giay
```

```
[('search', 0.358), ('cart', 0.512), ('search', 0.171), ('login', 0.433), ('cart', 0.621)]
```

Ưu tiên `mapValues` hơn `map(lambda kv: (kv[0], f(kv[1])))` sau các phép theo khoá.

`sample(withReplacement, fraction, seed)` — transformation · narrow

Tác dụng: lấy mẫu ngẫu nhiên; mỗi phần tử được chọn với xác suất `fraction`.

```
nums.sample(False, 0.5, seed=7).collect()
```

```
[1, 2, 2, 3]          <- 4/6 phan tu, khong phai dung 3 (50%)
```

Cùng `seed` và cùng cách chia partition → cùng kết quả. Dùng để chạy thử pipeline trên dữ liệu nhỏ.

`coalesce(n)` — transformation · narrow · chỉ giảm số partition

Tác dụng: gộp các partition kề nhau thành `n` partition, **không shuffle**.

```
nums.coalesce(1).glom().collect()
```

```
[[1, 2, 2, 3, 4, 5]]
```

Dùng sau khi lọc mạnh (bớt partition rỗng) hoặc trước khi ghi để có ít file hơn. Đo được nhanh 2,6× [slide 5.9].

`distinct()` — transformation · wide

Tác dụng: loại phần tử trùng lặp.

```
sorted(nums.distinct().collect())
```

```
[1, 2, 3, 4, 5]
```

Phải shuffle (gom giá trị giống nhau về một chỗ); thứ tự kết quả không đảm bảo — nên `sorted()` khi in.

`reduceByKey(f)` — transformation · wide · gộp cục bộ trước khi shuffle

Tác dụng: gộp các giá trị **cùng khoá** bằng hàm `f(a, b)`; kết quả một dòng mỗi khoá.

```
sorted(kv.reduceByKey(add).collect())                # tong latency theo endpoint
```

```
[('cart', 1133), ('login', 433), ('search', 529)]
```

`f` phải kết hợp và giao hoán. Là lựa chọn mặc định cho mọi phép tổng hợp theo nhóm.

`groupByKey()` — transformation · wide · không gộp cục bộ

Tác dụng: gom **tất cả** giá trị của mỗi khoá thành một tập.

```
sorted(kv.groupByKey().mapValues(list).collect())
```

```
[('cart', [512, 621]), ('login', [433]), ('search', [358, 171])]
```

Gửi mọi giá trị qua mạng; khoá nhiều giá trị dễ tràn bộ nhớ. Chỉ dùng khi thật sự cần cả danh sách (ví dụ sắp xếp sự kiện trong một phiên). Để tính tổng/đếm, dùng `reduceByKey` — ít hơn 116 lần byte shuffle [slide 5.5].

`join(other)` · `leftOuterJoin(other)` — transformation · wide · pair RDD

Tác dụng: ghép hai pair RDD theo khoá; `join` giữ khoá có ở cả hai bên, `leftOuterJoin` giữ mọi khoá bên trái.

```
sorted(kv.join(names).collect())
sorted(kv.leftOuterJoin(names).collect(), key=str)
```

```
[('cart', (512, 'Gio hang')), ('cart', (621, 'Gio hang')),
 ('search', (171, 'Tim kiem')), ('search', (358, 'Tim kiem'))]                 <- login,
    admin bi loai
[('cart', (512, 'Gio hang')), ('cart', (621, 'Gio hang')), ('login', (433, None)),
 ('search', (171, 'Tim kiem')), ('search', (358, 'Tim kiem'))]                 <- login giu
    lai, None
```

Khoá lặp ở hai bên → số dòng nhân lên. Bảng nhỏ → dùng broadcast thay cho `join` (Câu 14).

`repartition(n)` — transformation · wide · luôn shuffle

Tác dụng: chia lại dữ liệu thành đúng `n` partition (tăng hoặc giảm).

```
nums.repartition(3).glom().collect()
```

```
[[1, 2, 2], [3, 4, 5], []]      <- du lieu qua nho: repartition khong dam bao chia deu
```

Dùng khi cần *tăng* song song (file `.gz` chỉ có 1 partition) hoặc cân bằng dữ liệu lệch lớn. Để *giảm*, dùng `coalesce`.

15.2 Action

`count()` — action · trả về int

Tác dụng: đếm số phần tử; mỗi task đếm partition của mình, Driver cộng lại.

```
nums.count()
```

```
6
```

`take(n)` — action · trả về list n phần tử đầu

Tác dụng: lấy n phần tử **đầu tiên** theo thứ tự partition; chỉ chạy số task cần thiết (Câu 9).

```
nums.take(3)
```

```
[1, 2, 2]
```

Cách an toàn để "xem thử" dữ liệu lớn. Không ngẫu nhiên.

`takeSample(withReplacement, num, seed)` — action · trả về list đúng num phần tử

Tác dụng: lấy `num` phần tử ngẫu nhiên trên toàn RDD.

```
nums.takeSample(False, 3, seed=7)
```

```
[4, 1, 5]
```

Chạy ít nhất 2 job (đếm + lấy mẫu). Chỉ dùng với `num` nhỏ vì kết quả về Driver (Câu 12).

`reduce(f)` — action · trả về một giá trị

Tác dụng: gộp toàn bộ phần tử thành một giá trị bằng `f(a, b)`.

```
nums.reduce(add), nums.reduce(max)
```

```
(17, 5)
```

`f` phải kết hợp và giao hoán — phép chia, trừ, trung bình $(x + y)/2$ cho kết quả sai [slide 4.22–4.24].

`countByKey()` — action · trả về dict

Tác dụng: đếm số lần xuất hiện của mỗi khoá, trả `dict` về Driver.

```
sorted(kv.countByKey().items())
```

```
[('cart', 2), ('login', 1), ('search', 2)]
```

Chỉ khi số khoá nhỏ. Nhiều khoá → `kv.mapValues(lambda _: 1).reduceByKey(add)` để kết quả vẫn phân tán.

`collect()` — action · trả về list toàn bộ RDD

Tác dụng: kéo **mọi** phần tử về Driver thành list Python.

```
kv.collect()
```

```
[('search', 358), ('cart', 512), ('search', 171), ('login', 433), ('cart', 621)]
```

Chỉ dùng khi kết quả chắc chắn nhỏ (sau lọc, sau gom nhóm). Với dữ liệu lớn: Driver hết bộ nhớ [slide 5.6].

`saveAsTextFile(path)` — action · ghi ra thư mục

Tác dụng: mỗi executor ghi partition của mình thành một file văn bản trong thư mục `path`.

```
kv.reduceByKey(add).saveAsTextFile("/tmp/q9_out")
sorted(f for f in os.listdir("/tmp/q9_out") if not f.startswith("."))
open("/tmp/q9_out/part-00000").read().splitlines()
sc.textFile("/tmp/q9_out").collect()                # doc lai
```

```
['_SUCCESS', 'part-00000', 'part-00001']            <- 2 partition -> 2 file
["('search', 529)", "('cart', 1133)", "('login', 433)"]
["('search', 529)", "('cart', 1133)", "('login', 433)"]   <- doc lai chi con CHUOI: mat kieu
    du lieu
```

Báo lỗi nếu thư mục đã tồn tại. File `part-00001` ở đây rỗng: sau shuffle, cả 3 khoá rơi vào partition 0.

`foreach(f)` — action · không trả về gì

Tác dụng: chạy `f` trên từng phần tử **tại executor**, chỉ để tạo hiệu ứng phụ (ghi CSDL, gửi tin nhắn, cập nhật accumulator).

```
nums.foreach(lambda x: print(x))
```

Không có output trong ô notebook: `print` chạy trong Python worker, nội dung đi ra *terminal đang chạy Jupyter* (trên cluster: log của executor). Muốn thu kết quả về Driver, dùng `collect()` hoặc Accumulator. Với tài nguyên đắt (kết nối CSDL), dùng `foreachPartition`.

15.3 Bảng tóm tắt một trang

Hàm	Loại	Tác dụng	Ví dụ (kết quả trên dữ liệu mẫu)
`map(f)`	N	biến đổi từng phần tử	`x*10` → `[10,20,20,30,40,50]`
`filter(f)`	N	giữ phần tử thoả điều kiện	chẵn → `[2,2,4]`
`flatMap(f)`	N	biến đổi rồi trải phẳng	`split` → 5 từ
`mapValues(f)`	N	biến đổi giá trị, giữ khoá	ms → giây
`sample(...)`	N	mẫu theo xác suất	`0.5` → 4/6 phần tử
`coalesce(n)`	N	giảm partition, không shuffle	2 → 1 partition
`distinct()`	W	loại trùng	`[1,2,3,4,5]`
`reduceByKey(f)`	W	gộp theo khoá (có gộp cục bộ)	`('cart', 1133)`
`groupByKey()`	W	gom mọi giá trị theo khoá	`('cart', [512, 621])`
`join(o)`	W	ghép theo khoá	`('cart', (512, 'Gio hang'))`
`repartition(n)`	W	chia lại n partition	2 → 3 partition
`count()`	A	đếm	`6`
`take(n)`	A	n phần tử đầu	`[1, 2, 2]`
`takeSample(...)`	A	n phần tử ngẫu nhiên	`[4, 1, 5]`
`reduce(f)`	A	gộp thành một giá trị	`17`
`countByKey()`	A	đếm theo khoá → dict	`{'cart': 2, ...}`
`collect()`	A	toàn bộ RDD về Driver	list 5 cặp
`saveAsTextFile(p)`	A	ghi mỗi partition một file	`part-00000`, `part-00001`
`foreach(f)`	A	chạy `f` vì hiệu ứng phụ	(không trả về)

N = narrow transformation, W = wide transformation (shuffle), A = action.

PHẦN V

Thực thi pipeline, quan sát và tính đúng

Câu 16 – 19 · slide 4.16 – 4.25: pipeline hoàn chỉnh, Spark UI, log, tính đúng của `reduce`

Câu 16. Slide 4.16 có phải là tổng hợp các slide trước? Phần lý thuyết của pipeline nằm ở đâu?

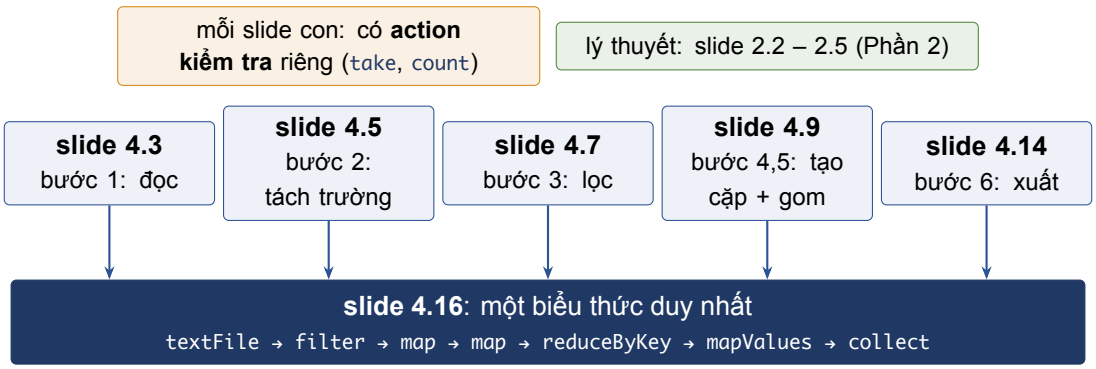
Trả lời ngắn

Đúng. Phần 4A được thiết kế theo đúng hướng "mỗi bước một slide con": slide 4.3, 4.5, 4.7, 4.9, 4.14 lần lượt hiện thực bước 1 2 3 4 5 6 của pipeline, mỗi slide có code chạy độc lập và output để kiểm tra. Slide 4.16 **nối các bước đó thành một biểu thức duy nhất**, với hai thay đổi có chủ đích: **đảo bước lọc lên trước bước tách trường** (tối ưu), và **bỏ các action kiểm tra ở giữa** (7 job còn 1 job). Mô tả lý thuyết của pipeline này nằm ở **Phần 2 của slide** (2.2–2.5), còn cách Spark thực thi nó nằm ở 3.16, 4.17, 4.18 và 5.11.

16.1 Ánh xạ từng bước: lý thuyết → slide con → dòng code trong 4.16

Bước	Lý thuyết (Phần 2)	Slide con	Dòng code trong 4.16	Loại
1	Đọc từng dòng log	4.3 (+ tài liệu 4.4)	`sc.textFile(..., 4)`	tạo RDD
3	Lọc chỉ giữ `ERROR`	4.7 (+ 4.8)	`.filter(lambda l: l.startswith("ERROR"))`	transformation, narrow
2	Tách 6 trường	4.5 (+ 4.6)	`.map(parse)`	transformation, narrow
4	Tạo cặp (endpoint, latency)	4.9 (+ 4.10)	`.map(lambda r: (r[3], (r[5], 1)))`	transformation, narrow
5	Gom nhóm, đếm, tính TB	4.9 (+ 4.10)	`.reduceByKey(...)` `.mapValues(...)`	transformation, **wide** + narrow
6	Xuất kết quả	4.14 (+ 4.15)	`.collect()`	**action**

Các slide 4.4, 4.6, 4.8, 4.10, 4.15 là *tài liệu API* của hàm dùng ở bước tương ứng; 4.11–4.13 là các hàm bổ trợ không nằm trong pipeline.



16.2 Hai điểm 4.16 *khác* với việc ghép nối đơn thuần

(1) Bước lọc được đưa lên trước bước tách trường

Trong Phần 2 và các slide con, thứ tự là 1→2→3 (tách trường rồi mới lọc — dễ hiểu). Ở 4.16 thứ tự là 1→3→2: lọc trên *chuỗi thô* bằng `startswith("ERROR")`, chỉ parse những dòng còn lại. Đây chính là **Nguyên tắc 1** [slide 5.3]. Kiểm chứng bằng Accumulator đếm số lần gọi `parse`:

```
so lan parse: parse -> filter = 6000 | filter -> parse = 366
```

(2) Các action kiểm tra ở giữa bị bỏ đi: 7 job còn 1 job

Mỗi slide con kết thúc bằng một action để *xem* kết quả bước đó. Chạy lại đúng như vậy rồi so với 4.16:

```
sc.setJobGroup("buoc", "buoc")
logs = sc.textFile(f"{DATA}/access.log", 4);           logs.count(); logs.take(2)   # 4.3
parsed = logs.map(parse);                              parsed.take(2)               # 4.5
errors = parsed.filter(lambda r: r[0] == "ERROR");     errors.count()               # 4.7
pairs = errors.map(lambda r: (r[3], (r[5], 1)));       pairs.take(3)                # 4.9
agg = pairs.reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1])); agg.collect()
kq1 = sorted(agg.mapValues(lambda s: (s[1], round(s[0] / s[1], 1))).collect())      # 4.14
print("tung buoc: so job =", len(sc.statusTracker().getJobIdsForGroup("buoc")))

sc.setJobGroup("gop", "gop")
kq2 = sorted(stats.collect())                                                       # 4.16
print("pipeline gop: so job =", len(sc.statusTracker().getJobIdsForGroup("gop")), "| giong
    nhau:", kq1 == kq2)
```

```
tung buoc: so job = 7
pipeline gop: so job = 1 | giong nhau: True
```

Mỗi action là một job chạy lại chuỗi từ đầu (lazy evaluation, không cache) — khi học, 7 job giúp *nhìn thấy* từng bước; khi chạy thật, chỉ cần 1 job. Đây là **Nguyên tắc 5.10**: đừng chen action kiểm tra vào giữa pipeline.

16.3 Phần lý thuyết của pipeline nằm ở đâu?

Slide	Nội dung	Vai trò đối với pipeline 4.16
2.2	Pipeline của bài toán xuyên suốt	**Định nghĩa 6 bước** 1–6 và khối lượng dữ liệu sau mỗi bước (6000 → 366 → 6)
2.3	Dữ liệu biến đổi qua từng bước	**Kiểu dữ liệu** ở đầu ra mỗi bước: chuỗi → tuple → cặp → (tổng, đếm) → (đếm, TB)
2.4	Hai loại bước theo phạm vi dữ liệu	Bước nào **narrow** (1–4), bước nào **wide** (5)
2.5	Mô tả biến đổi và yêu cầu kết quả	Bước nào là **transformation** (1–5), bước nào là **action** (6)
3.16	DAG — kế hoạch thực thi	DAG của chính pipeline này, cắt thành 2 stage
4.17, 4.18	Từ code tới job/stage/task; `toDebugString`	Spark thực thi 4.16 thế nào: 1 job, 2 stage, 4 + 4 task
4.22–4.25	Tính đúng của phép gộp	Vì sao bước 5 phải mang theo `(tổng, đếm)`
5.11	Áp dụng tổng hợp (TN10)	Cùng pipeline ở quy mô 1,2 triệu dòng: bản "ngây thơ" và bản tối ưu

Đọc theo trình tự nào?

Phần 2 (*cái gì*) → Phần 4A, slide con (*từng bước làm thế nào*) → 4.16 (*ghép lại*) → 4.17–4.18 (*Spark chạy nó ra sao*) → 5.11 (*làm cho nhanh*). Đây là cấu trúc "trên giấy trước, code sau" của cả bộ slide.

Câu 17. Spark UI và log là gì?

Trả lời ngắn

Đó là hai cách để **quan sát** một ứng dụng Spark đang chạy. **Spark UI** là **trang web** do chính Driver phục vụ (mặc định `http://localhost:4040`), hiển thị job, stage, task, bộ nhớ, cache bằng bảng và đồ thị — tốt để nhìn tổng thể. **Log** là **nhật ký văn bản** theo thời gian mà các thành phần Spark ghi ra (qua thư viện Log4j), mỗi dòng có thời điểm, mức độ, thành phần và thông điệp — tốt để truy vết chi tiết và tìm nguyên nhân lỗi. `applicationId` xuất hiện ở cả hai, dùng để đối chiếu.

17.1 Spark UI

Định nghĩa

Spark UI là một ứng dụng web được nhúng trong tiến trình Driver. Nó tồn tại **chỉ khi ứng dụng còn chạy** — sau `spark.stop()` trang biến mất. Cổng mặc định 4040; nếu bận (ví dụ đang mở hai notebook), Spark dùng 4041, 4042…

```
print(sc.uiWebUrl)
```

```
http://192.168.1.8:4040
```

Tab	Hiển thị	Dùng để trả lời câu hỏi
Jobs	mỗi action là một job; thời gian; *DAG Visualization*	"Code này chạy mấy job? Job nào lâu nhất?"
Stages	số task, thời gian từng task, **Shuffle Read/Write**, *skipped*	"Shuffle bao nhiêu byte? Có task nào chậm bất thường (skew)?"
Storage	RDD/DataFrame đang cache, số partition đã cache, dung lượng	"`cache()` có thật sự được lưu chưa?"
Environment	mọi cấu hình đang áp dụng	"`spark.driver.memory` có được đặt không?" (Câu 4)
Executors	bộ nhớ, số core, số task, thời gian GC của mỗi executor	"Executor nào quá tải? GC có nhiều không?"
SQL / DataFrame	kế hoạch truy vấn (Buổi 2)	"Catalyst đã tối ưu thế nào?"

REST API. Mọi thông tin trên UI cũng có dạng JSON tại `<uiWebUrl>/api/v1/...` — notebook `buoi1_toiuu.ipynb` dùng API này để đo số byte shuffle và số task:

```
api = f"{sc.uiWebUrl}/api/v1/applications"
json.load(urlopen(api))
json.load(urlopen(f"{api}/{sc.applicationId}/executors"))
```

```
[{"id": "local-1790550287594", "name": "LectureNote"}]
[{"id": "driver", "totalCores": 8, "totalTasks": 296, "maxMemory": 455501414}]
```

Chi tiết thú vị nối với Câu 2: `maxMemory` = 455.501.414 byte = 434,4 MiB, đúng bằng $(1024-300)\times 0{,}6$ MiB — tức là heap JVM 1 GB trừ 300 MB dự phòng, nhân tỉ lệ `spark.memory.fraction = 0.6` dành cho tính toán và cache.

Xem lại UI sau khi ứng dụng kết thúc

Bật ghi sự kiện: `.config("spark.eventLog.enabled", "true")` và `.config("spark.eventLog.dir", "file:///tmp/spark-events")`, rồi chạy **History Server** (`$SPARK_HOME/sbin/start-history-server.sh`, cổng 18080) để xem lại mọi ứng dụng đã chạy.

17.2 Log

Định nghĩa

Log là các dòng văn bản mà Driver và Executor ghi ra trong khi chạy, qua thư viện **Log4j 2**. Mỗi dòng có dạng:

`<ngày giờ> <MỨC ĐỘ> <Thành phần>: <thông điệp>`

Mức	Ý nghĩa
ERROR	lỗi làm hỏng task/job — luôn phải đọc
WARN	điều bất thường nhưng vẫn chạy được — mức mặc định trong notebook khoá học
INFO	diễn biến bình thường: job bắt đầu, stage nộp, task xong… — rất nhiều dòng
DEBUG, TRACE	chi tiết nội bộ, dùng khi gỡ lỗi Spark

Log nằm ở đâu?

- **Log của Driver** đi ra *stderr* của tiến trình Driver: trong Jupyter là **cửa sổ terminal đang chạy** `jupyter lab` (một phần có thể hiện dưới ô code dạng chữ nền đỏ).
- **Log của Executor**: ở chế độ local là cùng chỗ với Driver; trên cluster nằm trong thư mục làm việc của từng máy worker, hoặc lấy bằng `yarn logs -applicationId <id>`. Output của `print` trong `foreach` cũng nằm ở đây.
- Đổi mức log: `sc.setLogLevel("INFO")`. Cấu hình lâu dài: file `$SPARK_HOME/conf/log4j2.properties`.

Đọc một đoạn log thật

Ba dòng WARN xuất hiện mỗi khi khởi động trên máy này — đều **vô hại**:

```
WARN Utils: Your hostname, MacBook-Pro.local resolves to a loopback address: 127.0.0.1; using 192.168.1.8 instead (on
     interface en0)
WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where
     applicable
```

Sau `sc.setLogLevel("INFO")`, chạy pipeline `textFile(..., 4) → map → reduceByKey → collect`, log ghi lại toàn bộ vòng đời của job (đã rút gọn, giữ nguyên thông điệp):

```
06:05:12 INFO SparkContext: Starting job: collect at verify_note.py:247
06:05:12 INFO DAGScheduler: Got job 79 (collect at verify_note.py:247) with 4 output partitions
06:05:12 INFO DAGScheduler: Final stage: ResultStage 101 (collect at verify_note.py:247)
06:05:12 INFO DAGScheduler: Parents of final stage: List(ShuffleMapStage 100)
06:05:12 INFO DAGScheduler: Submitting 4 missing tasks from ShuffleMapStage 100 (PairwiseRDD[201] at reduceByKey ...)
06:05:12 INFO TaskSetManager: Starting task 0.0 in stage 100.0 (TID 296) (192.168.1.8, executor driver, partition 0,
     PROCESS_LOCAL, 8842 bytes)
06:05:12 INFO HadoopRDD: Input split: file:/.../access.log:0+62969
06:05:12 INFO HadoopRDD: Input split: file:/.../access.log:62969+62969
06:05:12 INFO TaskSetManager: Finished task 2.0 in stage 100.0 (TID 298) in 37 ms on 192.168.1.8 (executor driver)
     (1/4)
06:05:12 INFO DAGScheduler: ShuffleMapStage 100 (reduceByKey at verify_note.py:247) finished in 0.046 s
06:05:12 INFO DAGScheduler: Submitting 4 missing tasks from ResultStage 101 (PythonRDD[204] at collect ...)
06:05:12 INFO DAGScheduler: ResultStage 101 (collect at verify_note.py:247) finished in 0.023 s
06:05:12 INFO DAGScheduler: Job 79 finished: collect at verify_note.py:247, took 0.075403 s
```

Đọc đoạn log này, ta thấy lại gần như mọi khái niệm của buổi học:

- **1 action = 1 job**: `Starting job: collect`.
- **Wide transformation = 2 stage**: `ShuffleMapStage 100` (trước shuffle) là cha của `ResultStage 101`.
- **Số task = số partition** (Câu 9): `Submitting 4 missing tasks`, `Starting task 0.0 … partition 0`.
- **Partition của file** là các khoảng byte: `access.log:0+62969`, `62969+62969`… — 4 đoạn bằng nhau.
- **Local mode**: `executor driver` — executor nằm chung với Driver; `PROCESS_LOCAL` — dữ liệu ở ngay tiến trình chạy task.
- **Stage sau đợi stage trước**: `ResultStage 101` chỉ được nộp sau khi `ShuffleMapStage 100 finished`.

17.3 Spark UI hay log?

	Spark UI	Log
Dạng	trang web, bảng, đồ thị	dòng văn bản theo thời gian
Do ai tạo	Driver (web server nhúng)	Driver và mọi Executor (Log4j)
Tồn tại	khi ứng dụng chạy (hoặc qua History Server)	trong file/terminal, lâu dài
Mạnh ở	tổng quan, so sánh task, số liệu shuffle, cache	truy vết từng bước, **nguyên nhân lỗi** (stack trace)
Dùng khi	"Vì sao job chậm? Stage nào đắt?"	"Vì sao job hỏng? Chuyện gì xảy ra lúc 06:05?"

Quy trình gỡ lỗi đề xuất

1. Lỗi trong notebook (`Py4JJavaError`) → cuộn tới đoạn `PythonException: Traceback` để thấy lỗi Python gốc.
2. Chưa rõ → mở log (terminal Jupyter), tìm dòng `ERROR` đầu tiên quanh thời điểm lỗi.
3. Job chạy được nhưng chậm → mở Spark UI, tab *Stages*: xem shuffle bao nhiêu, task nào lâu hơn hẳn các task khác.

Câu 18. Spark UI qua hình chụp thực tế

Trả lời ngắn

Các hình dưới đây là **ảnh chụp thật** Spark UI của một ứng dụng mẫu tên `Buoi1-SparkUI-Demo`, chạy 5 job trên `access.log`. Mỗi hình có các số đánh dấu ❶ ❷… trỏ vào chỗ cần đọc, kèm giải thích bên dưới và liên hệ với các câu trước. Đọc lần lượt các hình là cách tốt nhất để thấy job, stage, task, shuffle, cache *bằng mắt*.

18.1 Ứng dụng dùng để chụp

Mỗi action được gắn một mô tả bằng `sc.setJobDescription(...)` để dễ nhận ra trên UI. Có thể chạy lại đoạn code này trong notebook rồi mở địa chỉ `sc.uiWebUrl` để tự quan sát.

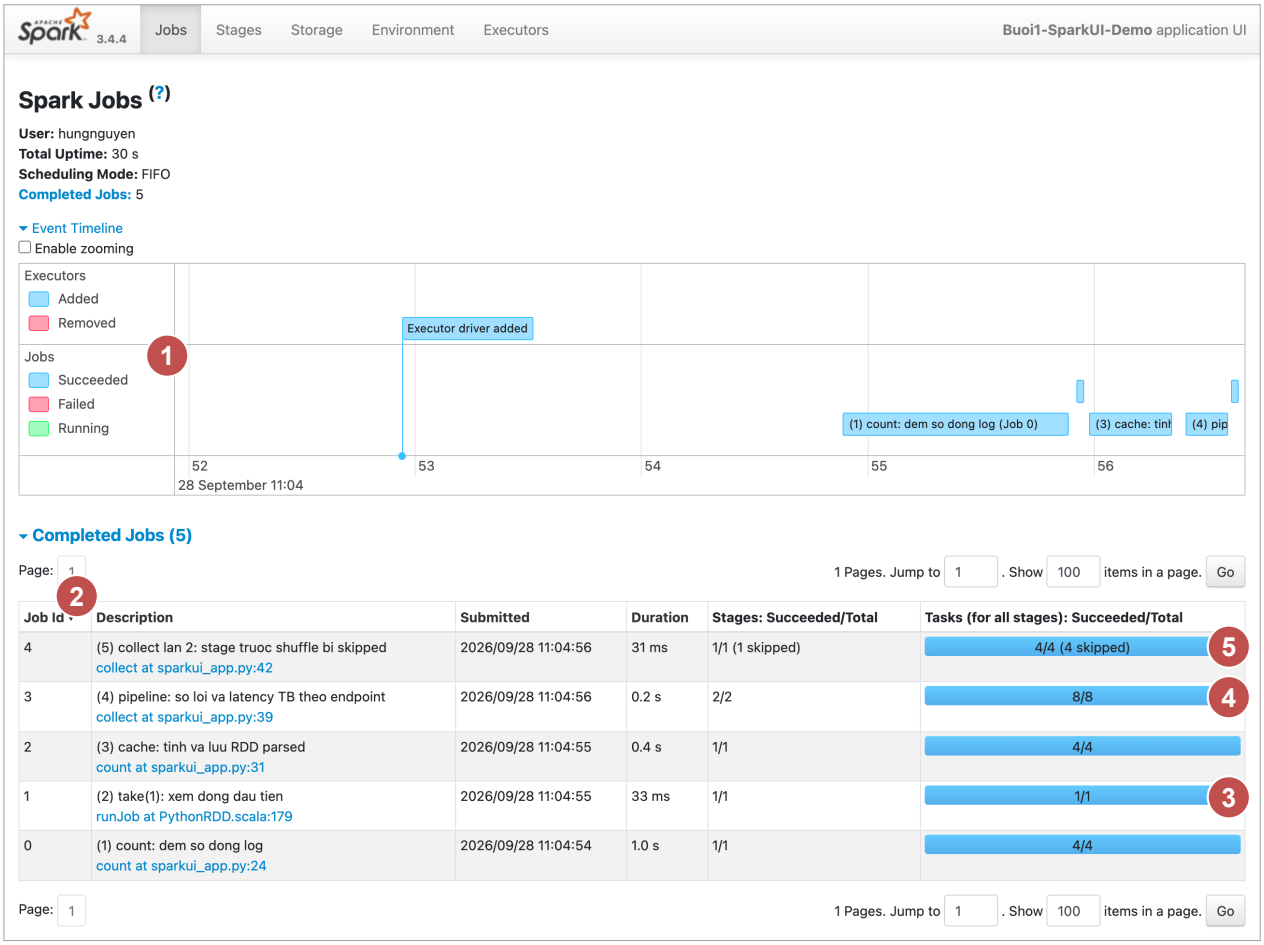
```
logs = sc.textFile(f"{DATA}/access.log", 4)

sc.setJobDescription("(1) count: dem so dong log");        logs.count()
sc.setJobDescription("(2) take(1): xem dong dau tien");    logs.take(1)

parsed = logs.map(parse).setName("parsed: access.log da parse").cache()
sc.setJobDescription("(3) cache: tinh va luu RDD parsed"); parsed.count()

stats = (logs.filter(lambda l: l.startswith("ERROR")).map(parse)
             .map(lambda r: (r[3], (r[5], 1)))
             .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))
             .mapValues(lambda s: (s[1], round(s[0] / s[1], 1))))
sc.setJobDescription("(4) pipeline: so loi va latency TB theo endpoint"); stats.collect()
sc.setJobDescription("(5) collect lan 2: stage truoc shuffle bi skipped"); stats.collect()
```

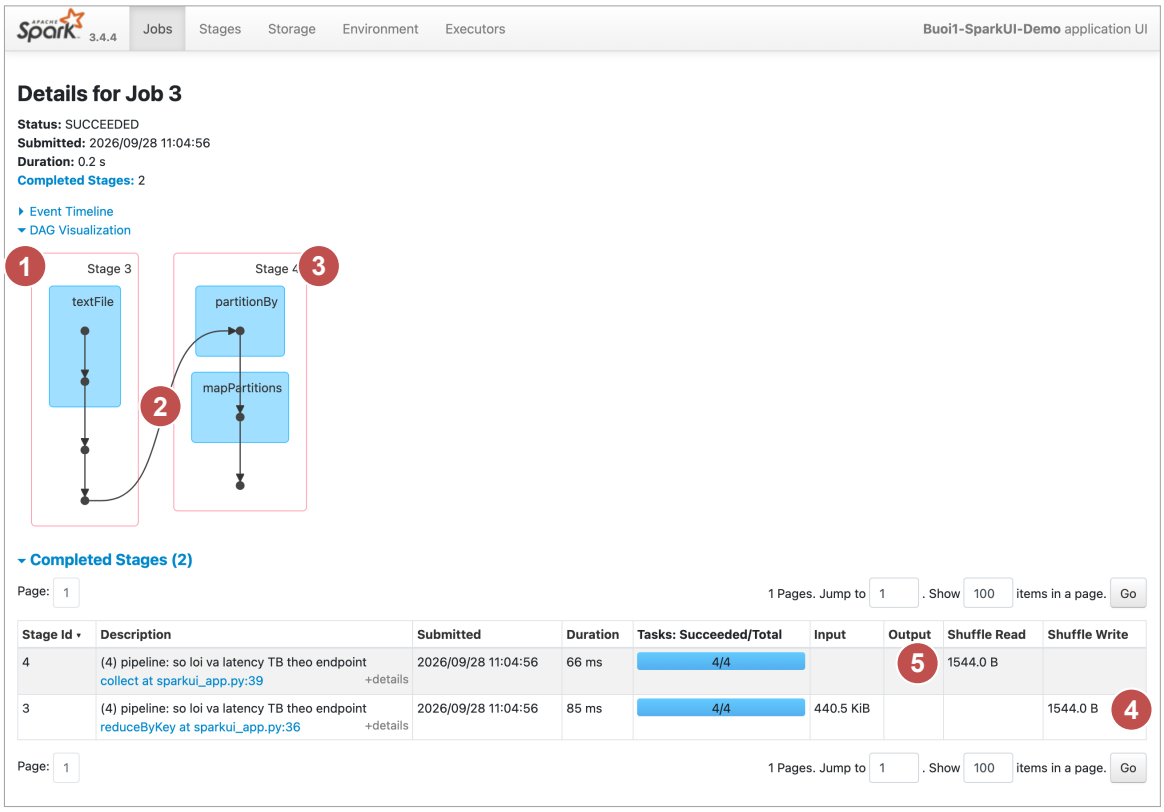
18.2 Tab Jobs — danh sách job của ứng dụng



1. **Event Timeline**: dòng thời gian — executor được thêm vào lúc nào, job nào chạy lúc nào, kéo dài bao lâu.
2. Cột **Description** hiện đúng mô tả đặt bằng `setJobDescription`; dòng xanh bên dưới là *action và dòng code* sinh ra job. **5 action → 5 job** (Job Id 0–4).
3. Job 1 — `take(1)`: chỉ **1/1 task**, vì chỉ cần partition đầu tiên (Câu 9).
4. Job 3 — pipeline: **2/2 stage**, **8/8 task** = 4 task trước shuffle + 4 task sau shuffle.
5. Job 4 — `collect` lần 2: **4/4 (4 skipped)**, stage 1/1 (1 skipped): stage trước shuffle không chạy lại.

18.3 Chi tiết một job — DAG Visualization

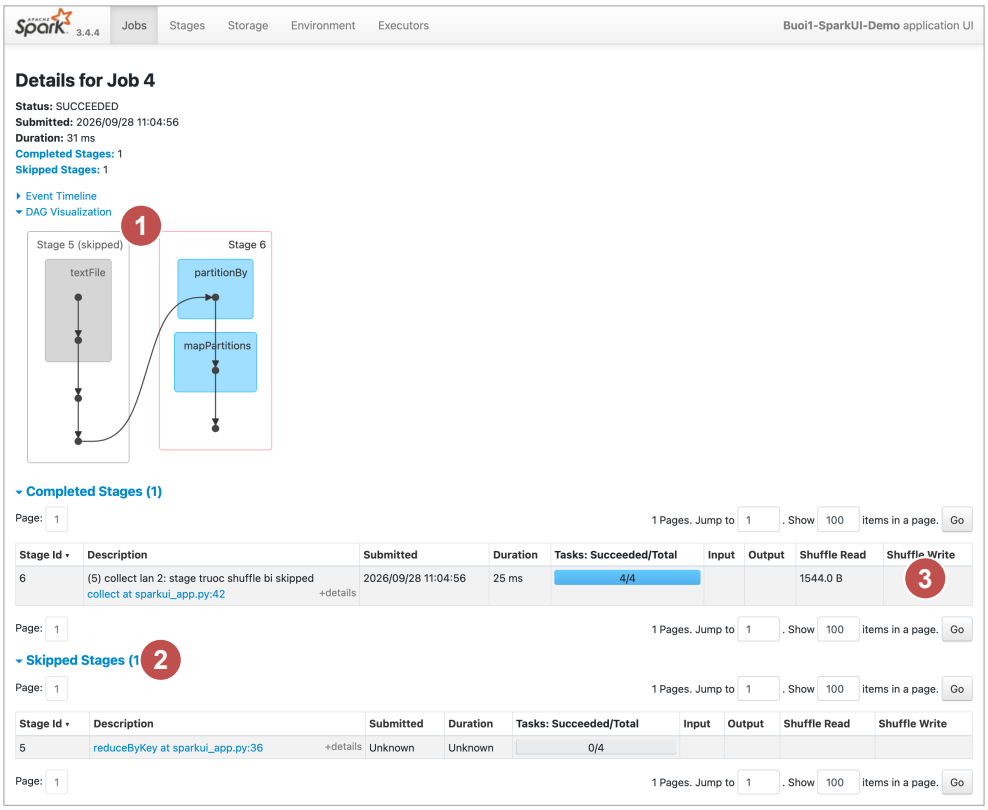
Bấm vào Job 3 (pipeline), mở mục *DAG Visualization*:



1. **Stage 3**: đọc file (`textFile`) rồi các bước Python *liền nhau* (filter, map, map, gộp cục bộ) — các chấm nối thẳng trong một khung = **pipelining** (Câu 9).
2. Đường cong nối hai stage = **shuffle** — ranh giới stage do `reduceByKey` tạo ra.
3. **Stage 4**: `partitionBy` (nhận dữ liệu shuffle) rồi `mapPartitions` (gộp toàn cục, `mapValues`).
4. Stage 3 **Shuffle Write** 1544 byte: sau khi gộp cục bộ, 6000 dòng chỉ còn vài bản ghi mỗi partition.
5. Stage 4 **Shuffle Read** đúng 1544 byte: đọc lại chính phần Stage 3 đã ghi.

18.4 Stage bị bỏ qua (skipped)

Job 4 gọi `collect()` lần nữa trên cùng RDD `stats`:

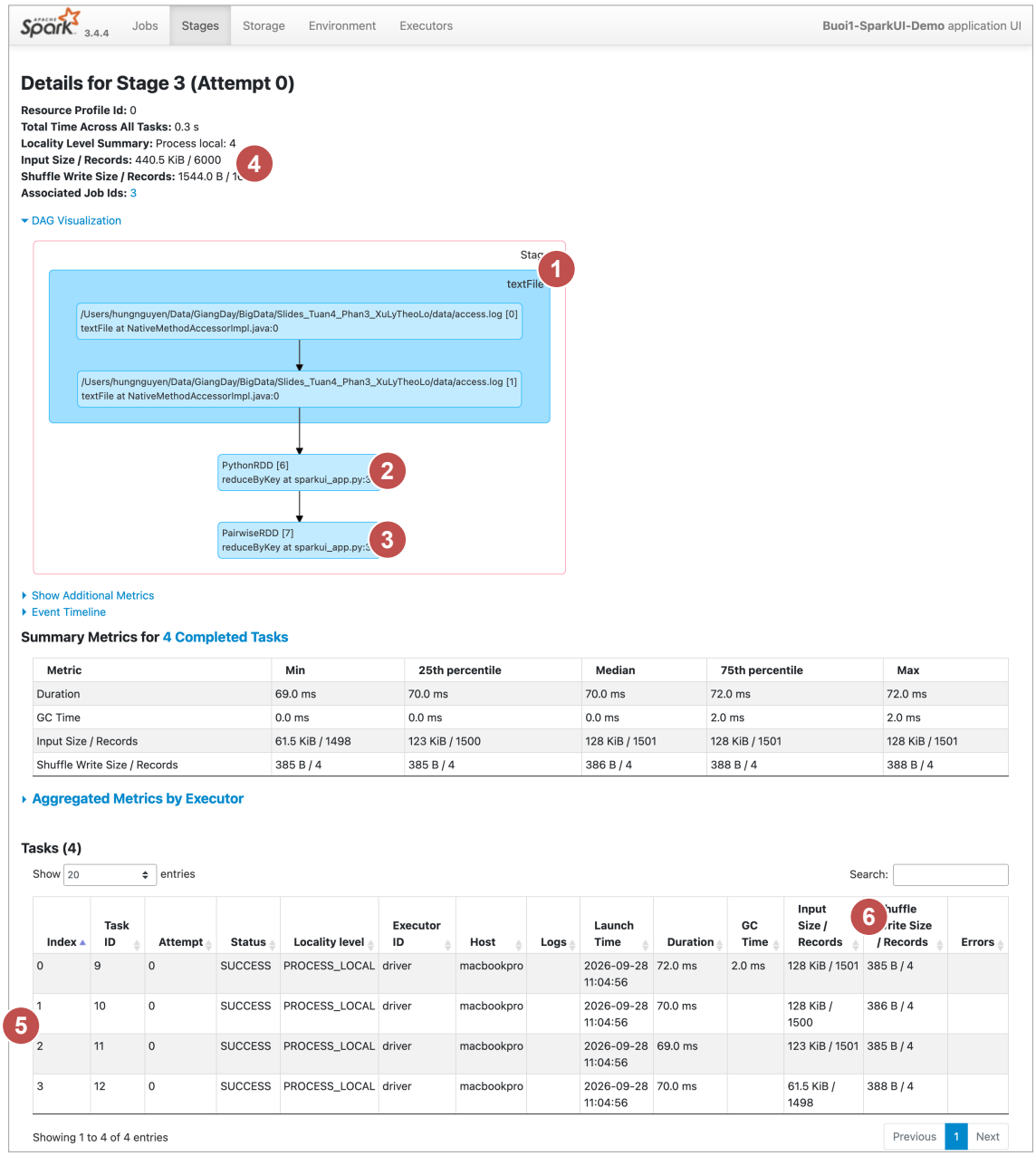


1. **Stage 5 (skipped)** tô xám: đây là stage đọc file + gộp cục bộ, nhưng kết quả shuffle của lần trước vẫn còn trên đĩa nên không cần chạy.
2. Bảng **Skipped Stages**: 0/4 task — không task nào được chạy.
3. Chỉ Stage 6 chạy: đọc lại 1544 byte shuffle đã có sẵn.

Khác với `cache()`: ở đây Spark tự dùng lại *file shuffle* mà không cần ta yêu cầu. Nhưng nếu job tạo RDD mới (gọi lại toàn bộ chuỗi `textFile(...)...`), Spark không nhận ra và sẽ chạy lại từ đầu.

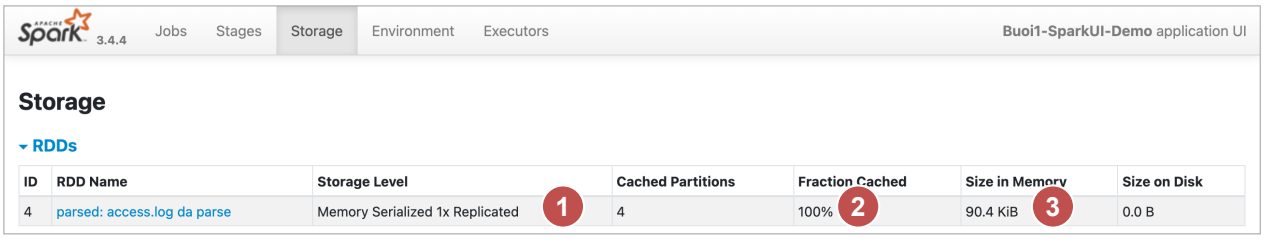
18.5 Chi tiết một stage — DAG bên trong và danh sách task

Bấm vào Stage 3:



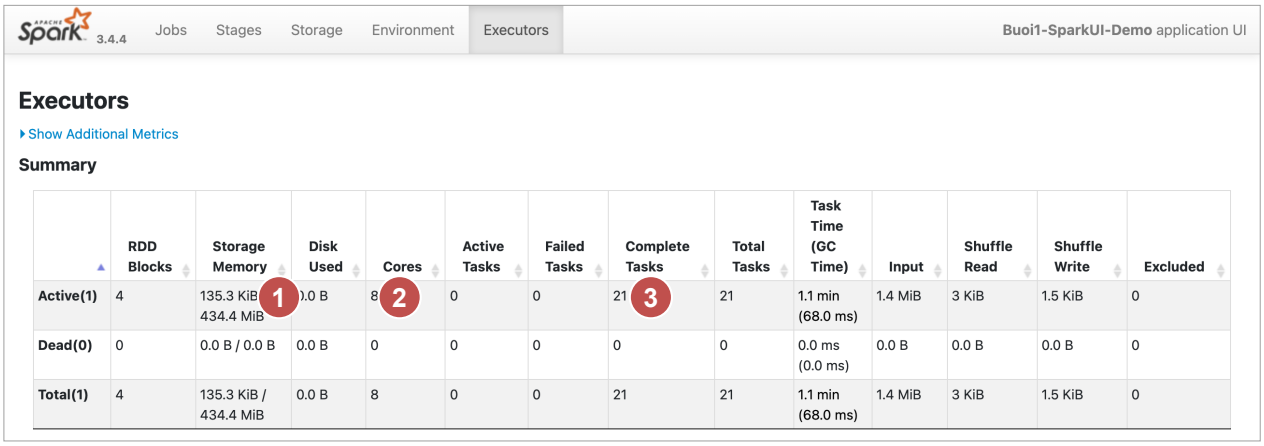
1. Khung `textFile`: `HadoopRDD [0]` và `MapPartitionsRDD [1]` — đọc file, tách dòng.
2. `PythonRDD [6]`: **một** nút duy nhất cho cả `filter` + `map` + `map` + gộp cục bộ — bằng chứng pipelining, giống hệt `toDebugString()` ở slide 4.18.
3. `PairwiseRDD [7]`: chuẩn bị cặp (khoá, giá trị) để ghi shuffle.
4. **Input Size / Records**: 6000 bản ghi được đọc; **Shuffle Write**: 1544 byte / 16 bản ghi.
5. **Bảng Tasks**: đúng **4 task**, cột *Index* 0, 1, 2, 3 — mỗi task một partition (Câu 9).
6. Cột *Input Size / Records*: 1501, 1500, 1501, 1498 dòng — khớp chính xác kết quả `mapPartitionsWithIndex` ở Câu 9. Các task đều chạy trên `executor driver`, locality `PROCESS_LOCAL`.

18.6 Tab Storage — RDD đang được cache



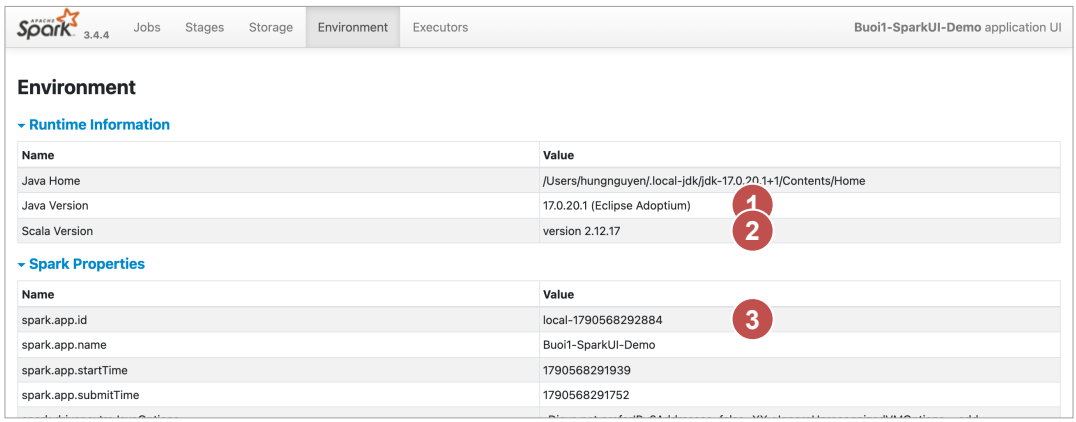
1. **Memory Serialized 1x Replicated**: PySpark lưu cache dưới dạng byte đã pickle (Câu 11), một bản sao.
2. **4 partition, 100%**: mọi partition đều đã được cache — cache theo từng partition (Câu 8).
3. **90,4 KiB** trong bộ nhớ. Tên hiển thị là tên đặt bằng `setName(...)` — nên đặt tên cho RDD cache để dễ tìm.

18.7 Tab Executors — tài nguyên của ứng dụng



1. **Storage Memory 135,3 KiB / 434,4 MiB**: đã dùng / tối đa. 434,4 MiB = $(1024 - 300) \times 0{,}6$ — chính là con số đã tính từ heap JVM ở Câu 17.
2. **Cores 8**: `local[*]` trên máy 8 luồng; chỉ có **một** executor (`driver`) — chế độ local.
3. **Complete Tasks 21** = 4 + 1 + 4 + 8 + 4 task của 5 job (4 task bị skip không được tính).

18.8 Tab Environment — cấu hình thực sự được áp dụng



1. **Java Version** 17.0.20.1 — JVM đang chạy Spark (Câu 2).
2. **Scala Version** 2.12.17 — Spark được viết bằng Scala, chạy trên JVM.

3. `spark.app.id`, `spark.app.name`: định danh của **Application** (Câu 6). Kiểm tra ở đây khi nghi ngờ một cấu hình không được áp dụng (Câu 4).

Cách tự chụp lại các hình này

Chạy đoạn code ở mục 18.1, mở `sc.uiWebUrl` trong trình duyệt. Các hình trong tài liệu được chụp tự động bằng Chrome ở chế độ headless (không giao diện) điều khiển qua Chrome DevTools Protocol, trong lúc ứng dụng vẫn đang chạy — vì Spark UI biến mất ngay khi gọi `spark.stop()`.

Câu 19. Vì sao `reduce` với phép chia cho kết quả khác nhau khi đổi số partition?

Trả lời ngắn

Vì `reduce` **không phải "một phép chia"** mà là **một chuỗi 5 phép chia hai ngôi**, và **cách chia partition quyết định cách đặt dấu ngoặc**. Phép chia không có tính kết hợp: $(a/b)/c = a/(bc)$ nhưng $a/(b/c) = ac/b$ — đổi chỗ ngoặc làm c **nhảy từ mẫu số lên tử số**. Với 3 partition, hai số $0{,}1$ và $0{,}2$ bị nhảy lên tử số, nên kết quả đổi từ 10 thành $0{,}004$. Với 2 partition cũng có hai số bị nhảy (5 và $0{,}2$), nhưng $5 \times 0{,}2 = 1$ nên *tình cờ* vẫn ra 10.

19.1 Một "phép chia" thực chất là năm phép chia

`reduce(f)` với 6 phần tử gọi `f` **5 lần**, mỗi lần với hai đối số. Chạy tuần tự từ trái sang phải (1 partition):

$$((((1 \div 2) \div 0{,}5) \div 0{,}1) \div 5) \div 0{,}2 = \frac{1}{2 \times 0{,}5 \times 0{,}1 \times 5 \times 0{,}2} = \frac{1}{0{,}1} = 10$$

Mọi phần tử sau phần tử đầu đều nằm ở **mẫu số**. Nhưng khi chia partition, Spark phải **nhóm lại** — mỗi partition tính riêng trước (tầng 1), rồi mới gộp các kết quả (tầng 2) [slide 4.23]:

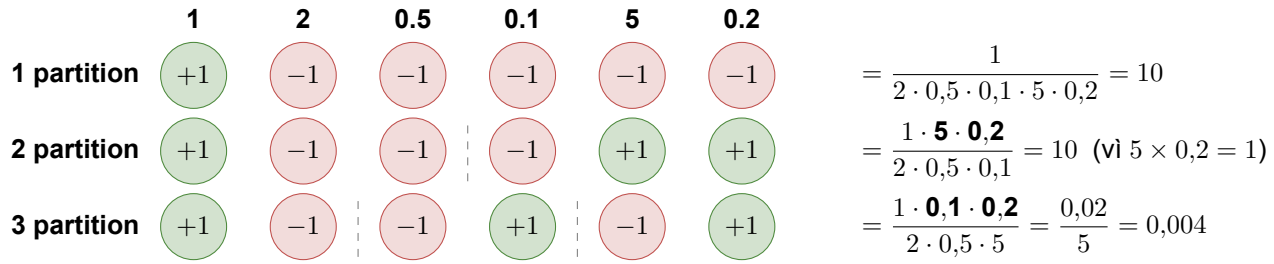
```
for p in (1, 2, 3, 6):
    rdd = sc.parallelize([1, 2, 0.5, 0.1, 5, 0.2], p)
    print(p, rdd.glom().collect(), "->", rdd.reduce(lambda x, y: x / y))
```

```
1 [[1, 2, 0.5, 0.1, 5, 0.2]]           -> 10.0    tang 1: [10.0]
2 [[1, 2, 0.5], [0.1, 5, 0.2]]         -> 10.0    tang 1: [1.0, 0.1]          -> 1.0 / 0.1
3 [[1, 2], [0.5, 0.1], [5, 0.2]]       -> 0.004   tang 1: [0.5, 5.0, 25.0]    -> 0.5 / 5.0 /
    25.0
6 [[1], [2], [0.5], [0.1], [5], [0.2]] -> 10.0    tang 1: moi partition 1 so -> giong fold
    trai
```

19.2 Theo dõi từng số: nằm ở tử số hay mẫu số?

Viết kết quả dưới dạng tích $\prod a_i^{e_i}$ với số mũ $e_i = +1$ (tử số) hoặc -1 (mẫu số). Quy luật khi gộp theo partition:

- Partition **đầu tiên**: phần tử đầu ở tử số, các phần tử sau ở mẫu số (như chia tuần tự).
- Mỗi partition **tiếp theo** cho ra một thương $q_k = b_1/(b_2 \cdots b_m)$, và ở tầng 2 thương này lại bị **chia vào**: $\ldots \div q_k = \ldots \times \dfrac{b_2 \cdots b_m}{b_1}$. Phần tử đầu b_1 vẫn ở mẫu số, nhưng **các phần tử còn lại** $b_2, \ldots, b_m$ **bị lật lên tử số**.



Số mũ được tính bằng chương trình theo đúng quy luật trên và nhân lại: 2 partition → 10.0, 3 partition → 0.004 — khớp kết quả của Spark. Đường nét đứt là ranh giới partition.

Vậy câu trả lời chính xác cho "tại sao": **số partition thay đổi thì tập các số nằm ở tử số thay đổi**. Phép toán "chỉ là phép chia" nhưng `reduce` đã âm thầm chuyển nó thành một biểu thức khác. Spark không làm sai — nó làm đúng điều mà tài liệu yêu cầu người viết hàm phải đảm bảo: *"The function should be commutative and associative so that it can be computed correctly in parallel."*

19.3 Vì sao Spark không giữ nguyên thứ tự trái sang phải?

Để chạy song song, mỗi executor phải tính **phần của mình** mà không đợi kết quả của partition phía trước. Nếu bắt buộc chia tuần tự từ trái sang phải, partition 2 phải chờ partition 1 xong — mất hoàn toàn lợi ích song song. Vì vậy `reduce` (và `reduceByKey`, `fold`, `aggregate`) **luôn** gộp theo cây: trong partition trước, giữa partition sau. Chỉ những phép toán cho cùng kết quả với *mọi* cách nhóm (kết hợp) và *mọi* thứ tự (giao hoán) mới an toàn.

19.4 Cách tính đúng: biến đổi về phép toán có tính kết hợp

$1 \div 2 \div 0{,}5 \div 0{,}1 \div 5 \div 0{,}2 = 1 \,/\, (2 \times 0{,}5 \times 0{,}1 \times 5 \times 0{,}2)$: phần tử đầu chia cho **tích** các phần tử còn lại. Phép nhân có tính kết hợp và giao hoán, nên tính tích bằng `reduce` an toàn, rồi chia **một lần** ở Driver — cùng mẫu thiết kế `(tổng, đếm)` của Câu 12 và slide 4.25:

```
from operator import mul
for p in (1, 2, 3, 6):
    rdd = sc.parallelize([1, 2, 0.5, 0.1, 5, 0.2], p)
    dau = rdd.first()
    tich = (rdd.zipWithIndex()                          # gan chi so 0, 1, 2, ... cho tung phan
    tu
                .filter(lambda t: t[1] > 0)             # bo phan tu dau
                .map(lambda t: t[0])
                .reduce(mul))                           # phep nhan: an toan voi moi cach chia
    print(p, "partition:", round(dau / tich, 6))
```

```
1 partition: 10.0
2 partition: 10.0
3 partition: 10.0
6 partition: 10.0
```

19.5 Mở rộng: ngay cả phép cộng số thực cũng không kết hợp tuyệt đối

Phép cộng số thực (`float`) chỉ kết hợp *về mặt toán học*; trên máy tính, mỗi phép cộng làm tròn, nên đổi cách nhóm có thể đổi vài chữ số cuối:

```
print((0.1 + 0.2) + 0.3, 0.1 + (0.2 + 0.3))
vals = [random.uniform(0, 1000) for _ in range(1_000_000)]       # random.seed(1)
```

```
for p in (1, 2, 3, 4, 8, 16):
    print(p, repr(sc.parallelize(vals, p).reduce(add)))
print("chinh xac (math.fsum):", repr(math.fsum(vals)))
```

```
0.6000000000000001 0.6
 1 500161973.4597767
 2 500161973.45979345
 3 500161973.4597986
 4 500161973.45980126
 8 500161973.4598006
16 500161973.4598004
chinh xac (math.fsum): 500161973.4598019     <- chenh lech lon nhat: 2.5e-05 (sai so tuong doi
    ~5e-14)
```

Hai mức độ "sai" rất khác nhau

- **Phép chia, trừ, trung bình** $(x + y)/2$: sai **về bản chất** — kết quả có thể lệch hàng nghìn lần (10 và 0,004). Không bao giờ dùng trong `reduce`.
- **Cộng số thực**: đúng về bản chất, chỉ lệch do làm tròn ở chữ số thứ 14 trở đi — chấp nhận được trong hầu hết phân tích. Nhưng đừng so sánh bằng `==` hai tổng số thực tính với số partition khác nhau; hãy so với sai số cho phép, hoặc dùng số nguyên (ví dụ lưu tiền theo đơn vị đồng, không dùng số thập phân).

Tài liệu tham khảo

1. M. Zaharia et al. *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*. NSDI 2012.
2. Apache Spark 3.4.4 Documentation: *RDD Programming Guide* (mục Understanding closures, Shared Variables), *Monitoring and Instrumentation* (Spark UI, REST API, History Server), *Configuration*.
3. PySpark 3.4.4 API Reference: `pyspark.RDD`, `pyspark.SparkContext`, `pyspark.sql.SparkSession`.
4. H. Wickham. *Tidy Data*. Journal of Statistical Software, 59(10), 2014.
5. Tài liệu khoá học: `buoi1_rdd.ipynb`, `buoi1_toiuu.ipynb`, `Slide_Buoi1_RDD_v2.pdf`.